

System Verilog Verification

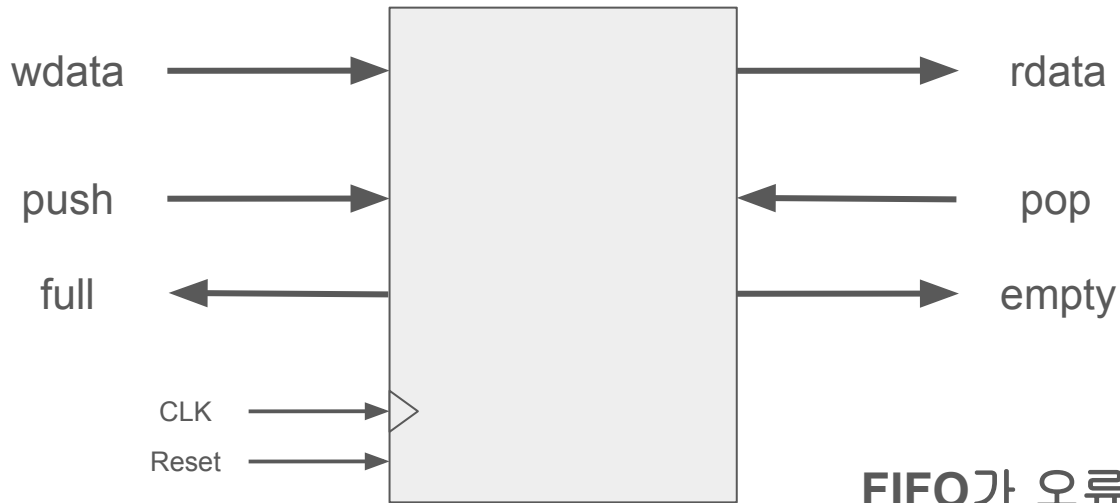
(UART/FIFO & STOPWATCH/CLOCK)

2026.03.03
김수빈, 하지훈

UART / FIFO

발표자 : 김수빈

FIFO



FIFO가 오류나기 쉬운 순간

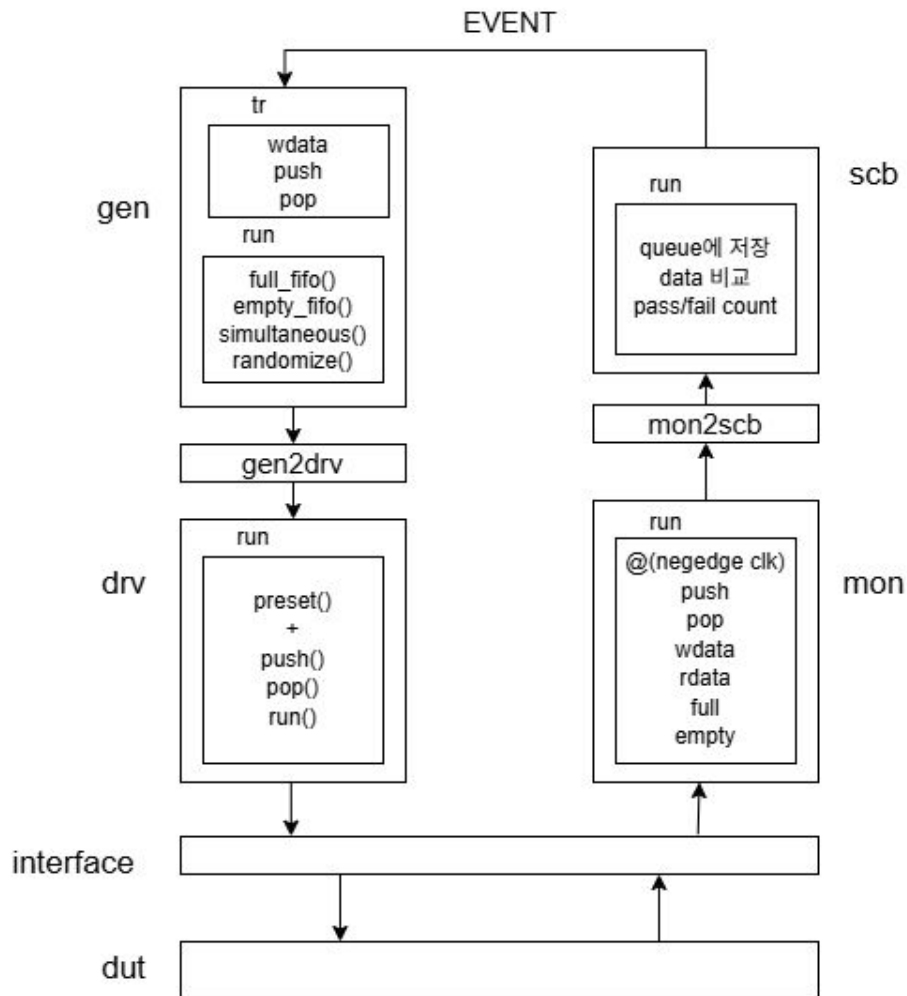
- FIFO가 완전히 비었을 때 (Empty)
- FIFO가 꽉 찼을 때 (Full)

FIFO 시나리오

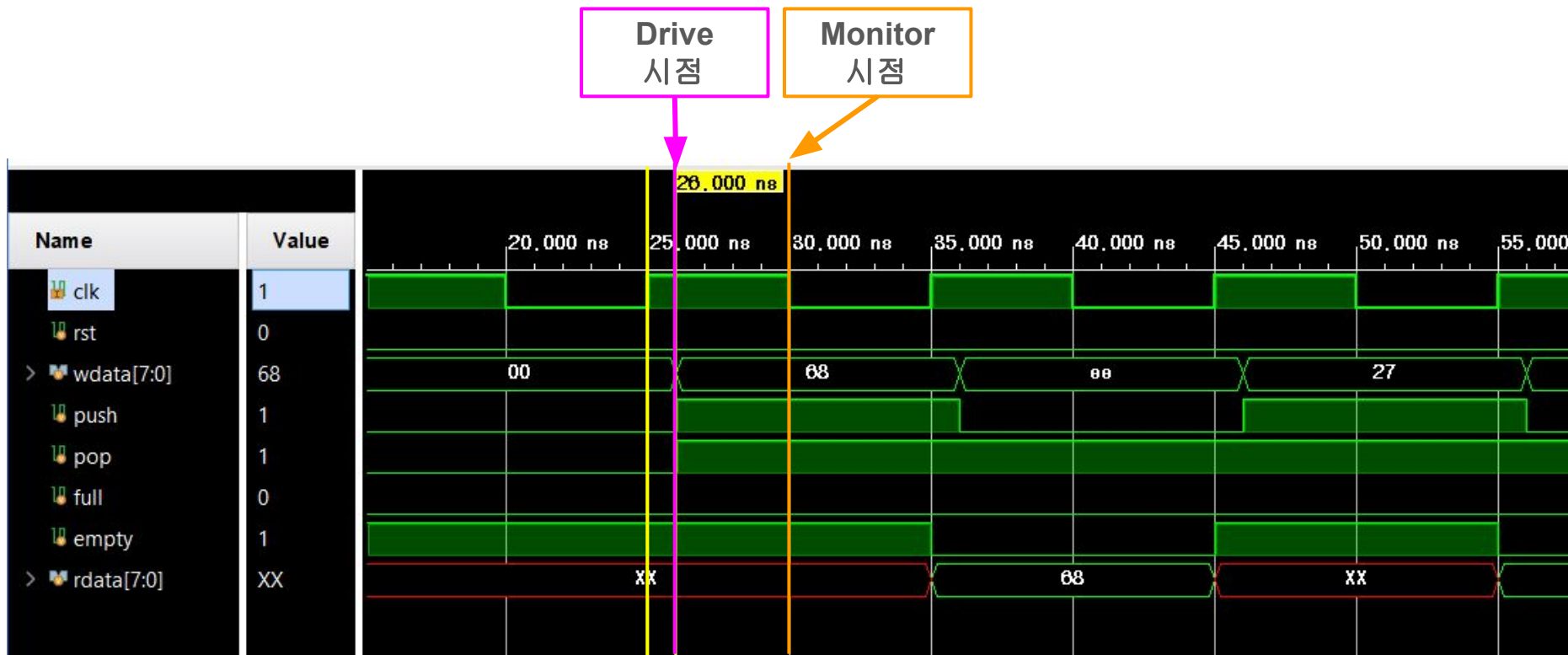
시나리오	내용
1	리셋 직후, Empty 상태 동시 입출력
2	FIFO 가득 채우기 : 16개 data 밀어 넣어서 Full 신호 띄우기
3	Full 상태 동시 입출력
4	FIFO 완전히 비우기
5	Random Test : 256회 실행 (8bit이므로)

Testbench Block (FIFO)

블록	동작
gen	full, empty, simultaneous, randomize 진행
drv	초기화 및 clk에 맞춰 push, pop, wdata 신호 전달
mon	Data Sampling
scb	비교 및 pass/fail count
interface	신호 연결



FIFO의 Timing



리셋 직후, Empty 상태에서 동시 입출력

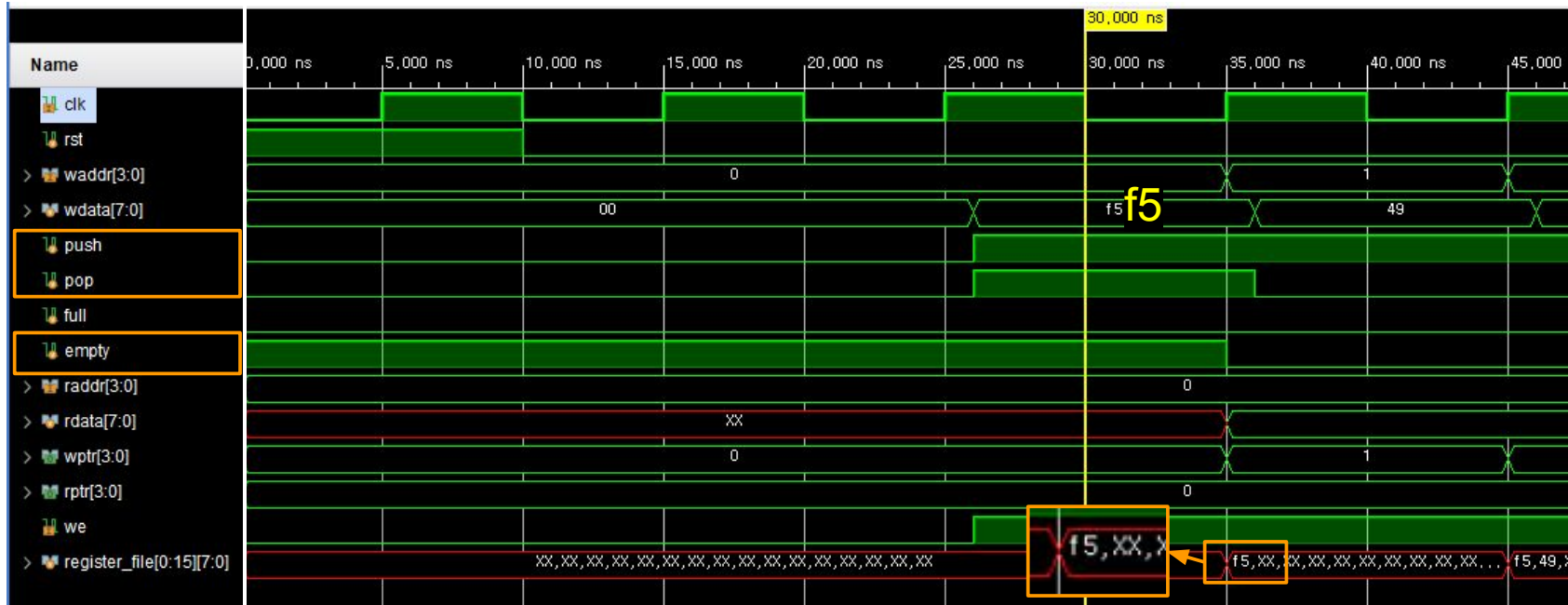
리셋 직후, FIFO가 비어있는 상태에서 push, pop 동시 발생
설계에 따르면, Data를 즉시 출력하지 않고 0번지 메모리에
저장되어야 함

=== [Scenario 1] Empty Simultaneous ===

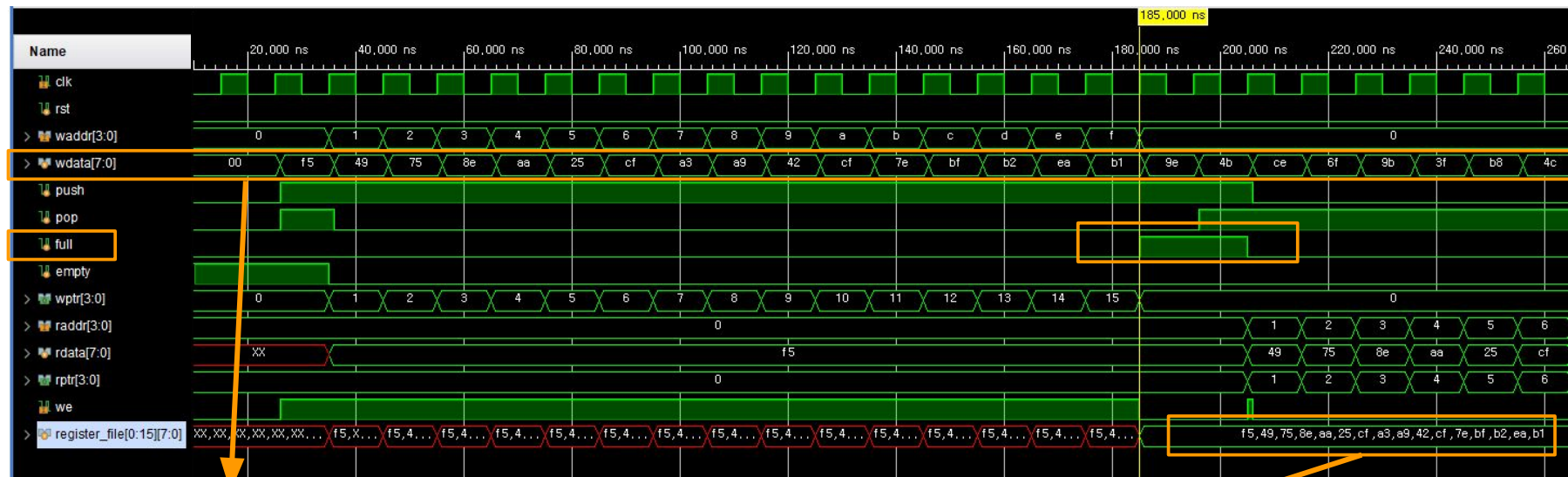
20000 : [gen_S4_simul] push = 1, wdata = f5, full = x, pop = 1, rdata = xx, empty = x

26000 : [drv] push = 1, wdata = f5, full = x, pop = 1, rdata = xx, empty = x

30000 : [mon] push = 1, wdata = f5, full = 0, pop = 1, rdata = xx, empty = 1



FIFO 가득 채우기 : 16개 Data 밀어 넣어서 Full = 1



register에 모두 저장되었을 때, Full = 1

Full 상태에서 동시 입출력

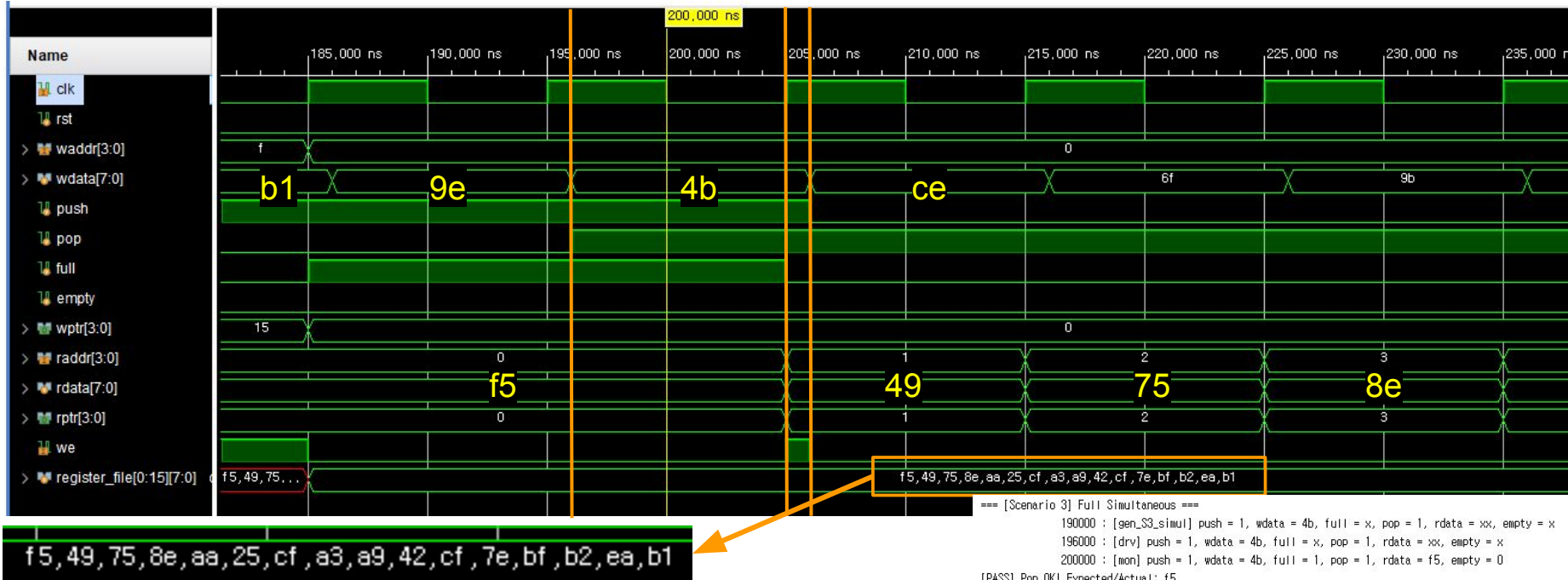
Full이 아닐 때만, Push가 작용하도록 설계

새로 들어온 Data는 저장되지 않음
원래 나갈 차례인 Data 출력

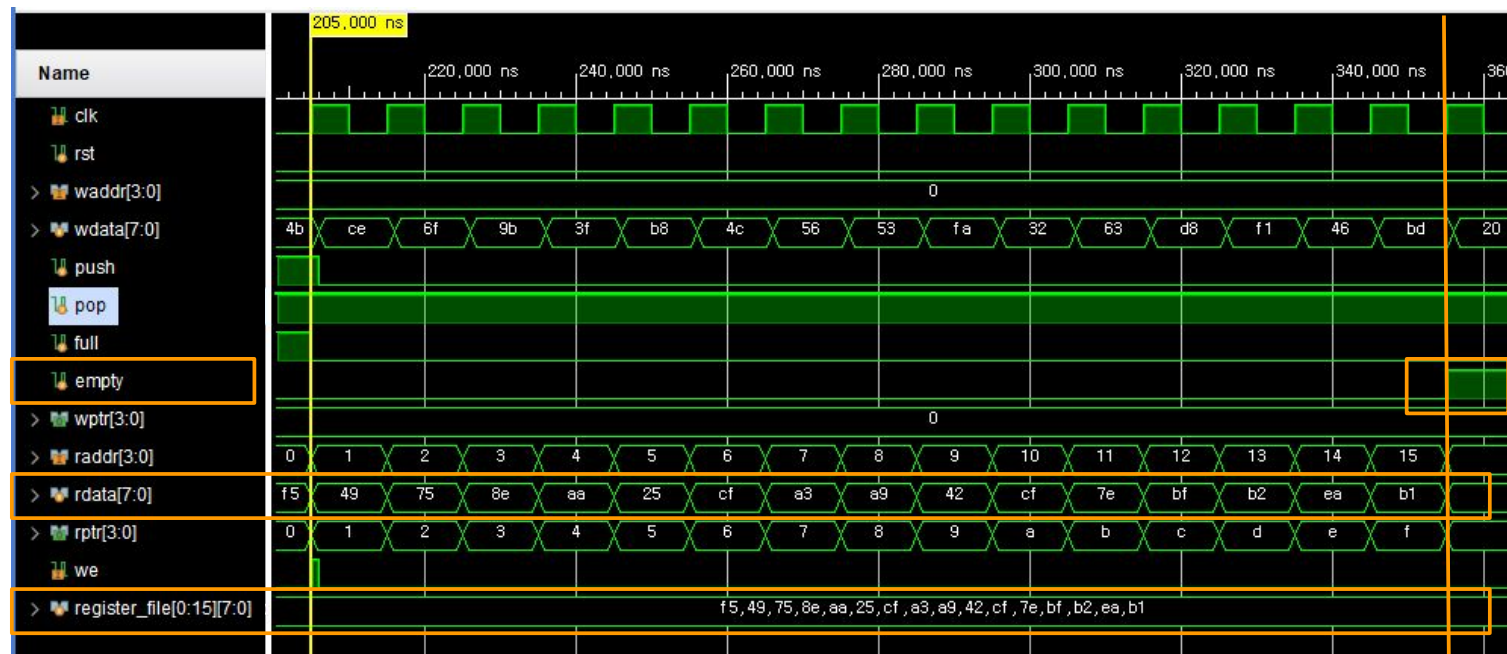
Push = 1, Pop = 1일 때, 들어온 Data = 4b 는

register에 저장 안됨

원래 나가야 하는 f5 출력



FIFO 완전히 비우기



FIFO 완전히 비우기

49,75,8e,aa,25,cf,a3,a9,42,cf,7e,bf,b2,ea,b1		

=== [Scenario 4] Empty FIFO ===

```
200000 : [gen_S2_empty] push = 0, wdata = ce, full = x, pop = 1, rdata = xx, empty = x
206000 : [drv] push = 0, wdata = ce, full = x, pop = 1, rdata = xx, empty = x
210000 : [mon] push = 0, wdata = ce, full = 0, pop = 1, rdata = 49, empty = 0
```

[PASS] Pop OK! Expected/Actual: 49

```
210000 : [gen_S2_empty] push = 0, wdata = 6f, full = x, pop = 1, rdata = xx, empty = x
216000 : [drv] push = 0, wdata = 6f, full = x, pop = 1, rdata = xx, empty = x
220000 : [mon] push = 0, wdata = 6f, full = 0, pop = 1, rdata = 75, empty = 0
```

[PASS] Pop OK! Expected/Actual: 75

```
220000 : [gen_S2_empty] push = 0, wdata = 9b, full = x, pop = 1, rdata = xx, empty = x
226000 : [drv] push = 0, wdata = 9b, full = x, pop = 1, rdata = xx, empty = x
230000 : [mon] push = 0, wdata = 9b, full = 0, pop = 1, rdata = 8e, empty = 0
```

[PASS] Pop OK! Expected/Actual: 8e

```
230000 : [gen_S2_empty] push = 0, wdata = 3f, full = x, pop = 1, rdata = xx, empty = x
236000 : [drv] push = 0, wdata = 3f, full = x, pop = 1, rdata = xx, empty = x
240000 : [mon] push = 0, wdata = 3f, full = 0, pop = 1, rdata = aa, empty = 0
```

[PASS] Pop OK! Expected/Actual: aa

```
240000 : [gen_S2_empty] push = 0, wdata = b8, full = x, pop = 1, rdata = xx, empty = x
246000 : [drv] push = 0, wdata = b8, full = x, pop = 1, rdata = xx, empty = x
250000 : [mon] push = 0, wdata = b8, full = 0, pop = 1, rdata = 25, empty = 0
```

[PASS] Pop OK! Expected/Actual: 25

```
250000 : [gen_S2_empty] push = 0, wdata = 4c, full = x, pop = 1, rdata = xx, empty = x
256000 : [drv] push = 0, wdata = 4c, full = x, pop = 1, rdata = xx, empty = x
260000 : [mon] push = 0, wdata = 4c, full = 0, pop = 1, rdata = cf, empty = 0
```

[PASS] Pop OK! Expected/Actual: cf

```
260000 : [gen_S2_empty] push = 0, wdata = 56, full = x, pop = 1, rdata = xx, empty = x
266000 : [drv] push = 0, wdata = 56, full = x, pop = 1, rdata = xx, empty = x
270000 : [mon] push = 0, wdata = 56, full = 0, pop = 1, rdata = a3, empty = 0
```

[PASS] Pop OK! Expected/Actual: a3

```
270000 : [gen_S2_empty] push = 0, wdata = 53, full = x, pop = 1, rdata = xx, empty = x
276000 : [drv] push = 0, wdata = 53, full = x, pop = 1, rdata = xx, empty = x
280000 : [mon] push = 0, wdata = 53, full = 0, pop = 1, rdata = a9, empty = 0
```

[PASS] Pop OK! Expected/Actual: a9

```
280000 : [gen_S2_empty] push = 0, wdata = fa, full = x, pop = 1, rdata = xx, empty = x
286000 : [drv] push = 0, wdata = fa, full = x, pop = 1, rdata = xx, empty = x
290000 : [mon] push = 0, wdata = fa, full = 0, pop = 1, rdata = 42, empty = 0
```

[PASS] Pop OK! Expected/Actual: 42

```
290000 : [gen_S2_empty] push = 0, wdata = 32, full = x, pop = 1, rdata = xx, empty = x
296000 : [drv] push = 0, wdata = 32, full = x, pop = 1, rdata = xx, empty = x
300000 : [mon] push = 0, wdata = 32, full = 0, pop = 1, rdata = cf, empty = 0
```

[PASS] Pop OK! Expected/Actual: cf

```
300000 : [gen_S2_empty] push = 0, wdata = 63, full = x, pop = 1, rdata = xx, empty = x
306000 : [drv] push = 0, wdata = 63, full = x, pop = 1, rdata = xx, empty = x
310000 : [mon] push = 0, wdata = 63, full = 0, pop = 1, rdata = 7e, empty = 0
```

[PASS] Pop OK! Expected/Actual: 7e

```
310000 : [gen_S2_empty] push = 0, wdata = d8, full = x, pop = 1, rdata = xx, empty = x
316000 : [drv] push = 0, wdata = d8, full = x, pop = 1, rdata = xx, empty = x
320000 : [mon] push = 0, wdata = d8, full = 0, pop = 1, rdata = bf, empty = 0
```

[PASS] Pop OK! Expected/Actual: bf

```
320000 : [gen_S2_empty] push = 0, wdata = f1, full = x, pop = 1, rdata = xx, empty = x
326000 : [drv] push = 0, wdata = f1, full = x, pop = 1, rdata = xx, empty = x
330000 : [mon] push = 0, wdata = f1, full = 0, pop = 1, rdata = b2, empty = 0
```

[PASS] Pop OK! Expected/Actual: b2

```
330000 : [gen_S2_empty] push = 0, wdata = 46, full = x, pop = 1, rdata = xx, empty = x
336000 : [drv] push = 0, wdata = 46, full = x, pop = 1, rdata = xx, empty = x
340000 : [mon] push = 0, wdata = 46, full = 0, pop = 1, rdata = ea, empty = 0
```

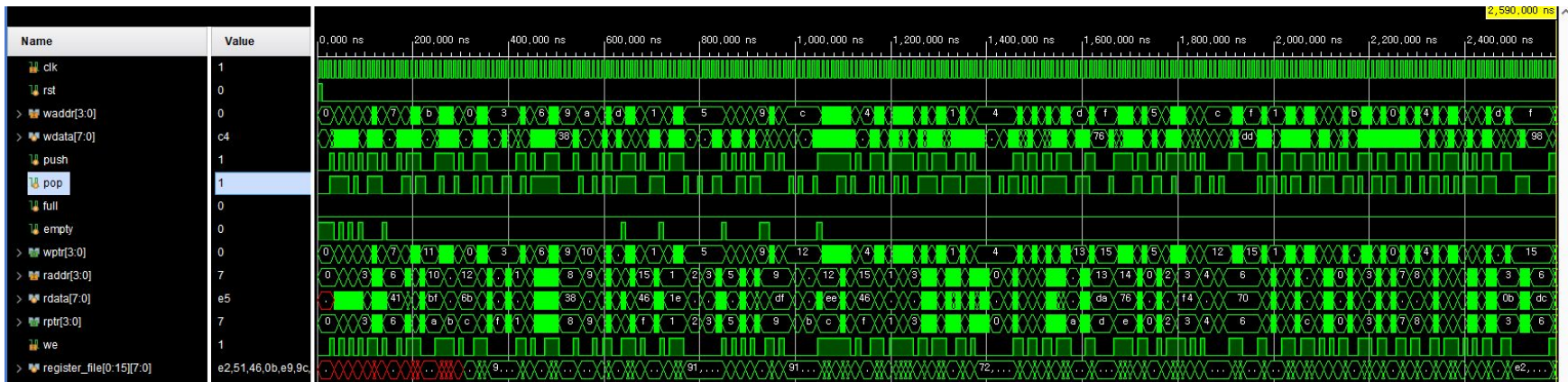
[PASS] Pop OK! Expected/Actual: ea

```
340000 : [gen_S2_empty] push = 0, wdata = bd, full = x, pop = 1, rdata = xx, empty = x
346000 : [drv] push = 0, wdata = bd, full = x, pop = 1, rdata = xx, empty = x
350000 : [mon] push = 0, wdata = bd, full = 0, pop = 1, rdata = b1, empty = 0
```

[PASS] Pop OK! Expected/Actual: b1

```
350000 : [gen_S2_empty] push = 0, wdata = 20, full = x, pop = 1, rdata = xx, empty = x
356000 : [drv] push = 0, wdata = 20, full = x, pop = 1, rdata = xx, empty = x
360000 : [mon] push = 0, wdata = 20, full = 0, pop = 1, rdata = f5, empty = 1
```

Random Test : 256회 실행 (8bit이므로)



[TEST REPORT] SIMULATION FINISHED!

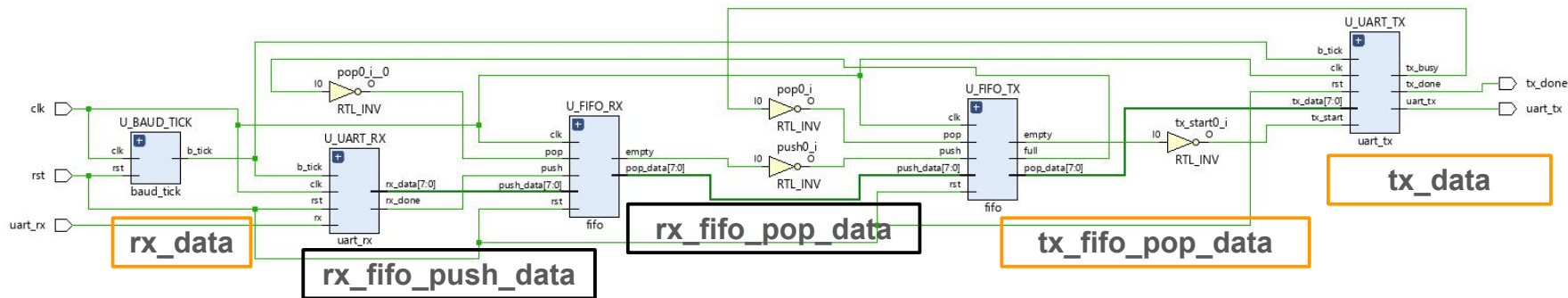
Generated wdata Count : 256
Actual Pop Count : 119
Total PASS Count : 119
Total FAIL Count : 0
Result : PERFECT SUCCESS!

256개의 랜덤 transaction으로 FIFO Test 수행

→ 실제 발생한 119개의 pop 동작을 검증

모든 데이터가 완벽하게 일치

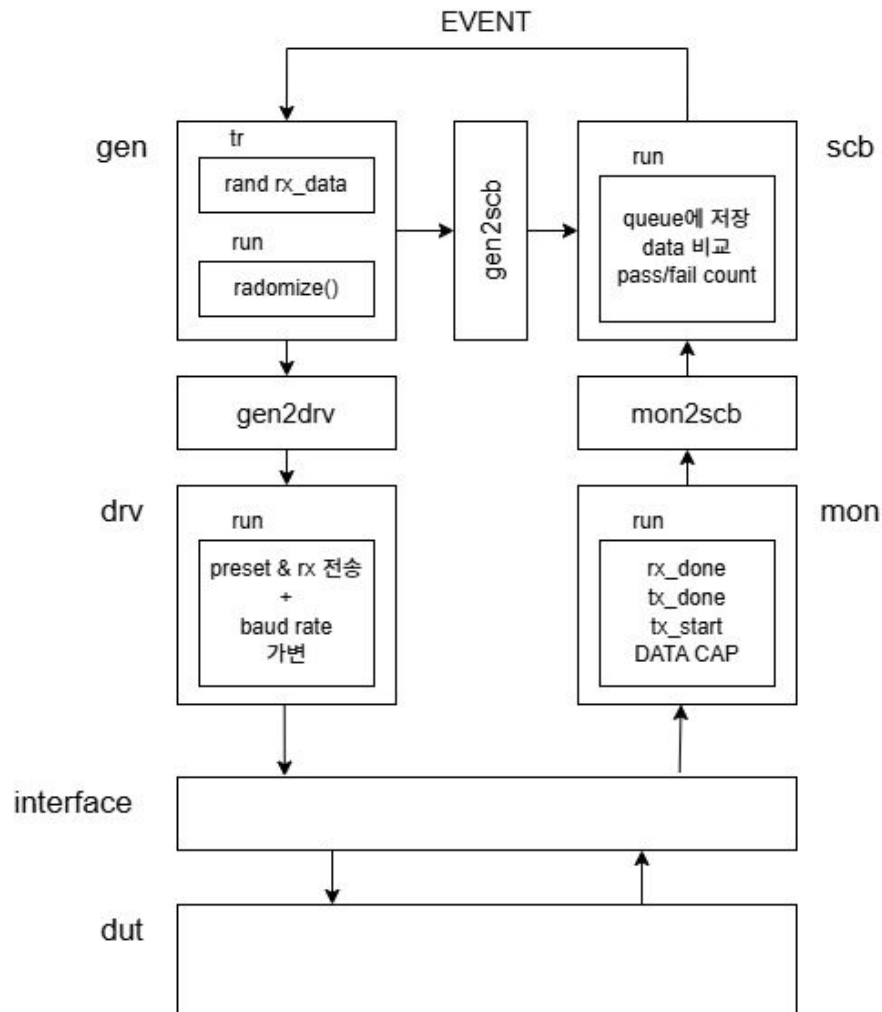
UART + FIFO Simulation 시나리오



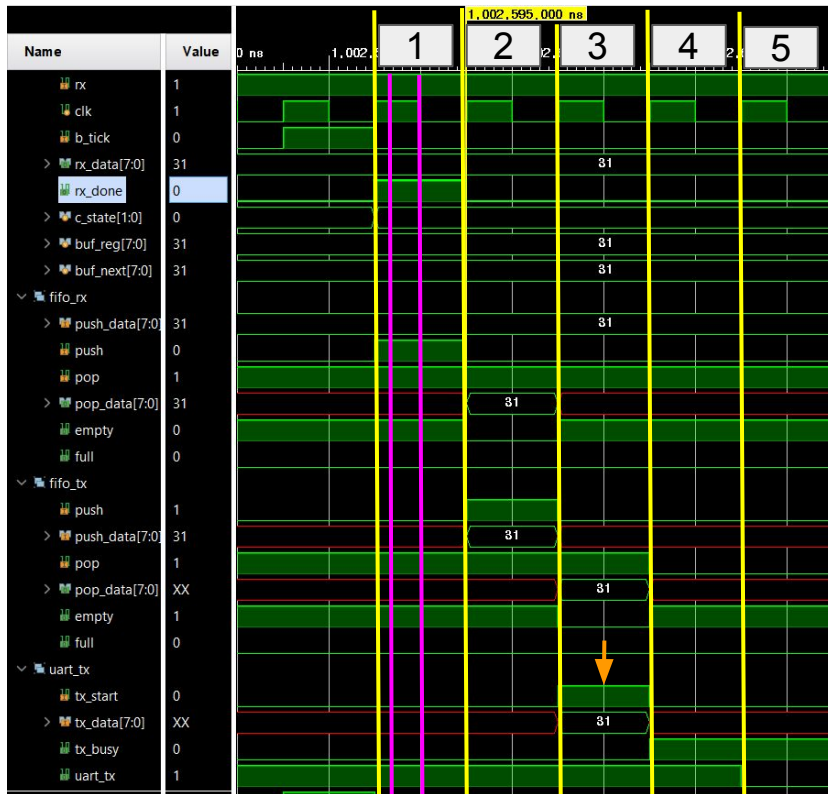
목적	비교 대상
random 생성된 RX 값이 FIFO를 제대로 통과했는지 확인	UART RX 입력 = tx_fifo_pop_data
random 생성된 RX 값이 UART TX를 제대로 통과했는지 확인	UART RX 입력 = tx_data
BAUD RATE 한계 범위 확인	UART RX 입력 = tx_data

Testbench Block (UART)

블록	동작
gen	랜덤 rx data 생성
drv	UART 비트 전송, 리셋 제어
mon	Data Sampling
scb	비교 및 pass/fail count
interface	신호 연결



UART RX - FIFO RX - FIFO TX - UART TX 의 Timing



[1]

rx_done = 1
fifo_rx_push = 1
fifo_rx_pop = 1

[2]

fifo_rx_pop data = 31
fifo_rx_empty = 0

fifo_tx push = 1
fifo_tx_push data = 31
fifo_tx_pop = 1

[3]

fifo_tx_pop data = 31
fifo_rx_empty = 1
fifo_tx_empty = 0
uart_tx_start = 1
uart_tx_data = 31

[4]

fifo_tx_empty = 1
uart_tx_busy = 1

[5]

uart_tx = 0

tx_done = 1

[drive 시점] posedge 이후, 1ns 지난 다음
[Monitor 시점] 신호가 1로 변한 신호 기준, negedge

TX Data 3,124.8us 이후 출력 → 값 비교 조건 추가



```
if (tr.tx_done) begin
    tr.display("tx_data_compare");
    if (uf_queue.size() > 0) begin
        // Act 값이 xx가 아닐 때만 queue에서 꺼내서 비교
        if (tr.tx_data != 8'hxx) begin
            compare_data = uf_queue.pop_front();
            compared_cnt++;
            if (compare_data === tr.tx_data) begin
                $display("PASS!!! (Exp: %h, Act: %h)",
                    compare_data, tr.tx_data);
                FINAL_pass_cnt++;
            end
        end
    end
end
```

[tx_data_compare] rx_data = 04, rx_done = x, tx_data = xx, tx_done = 1,

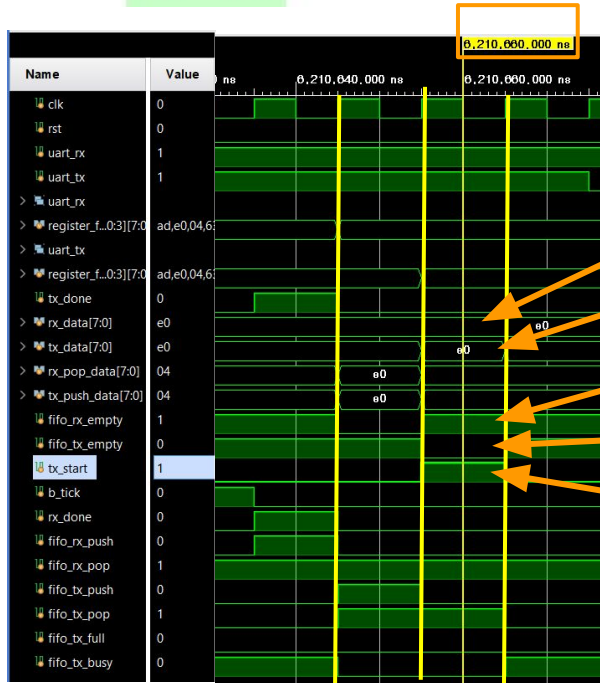
[tx_data_compare] rx_data = 56, rx_done = x, tx_data = b2, tx_done = 1,

```
begin
    wait (scb.compared_cnt >= (i - 3));
    $display("\n%t : SUCCESS", $time);
end
```


RX Data = FIFO TX POP Data

```
6210660000 : [FIFO_DATA_MATCH] FIFO trans Success! (Data: e0) PASS!!!
```

```
6210660000 : [FIFO_DATA_MATCH] fifo_rx_empty = 1, fifo_tx_empty = 0
```



rx_data = e0

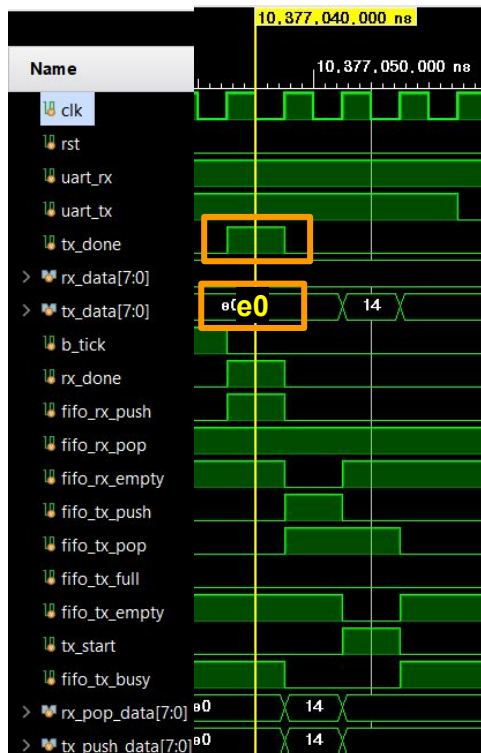
tx_data = fifo_tx_data = e0

fifo_rx_empty = 1

fifo_tx_empty = 0

tx_start = 1

TX Data = Compare Data : PASS



```
9375046000 : [drv] rx_data = 14, rx_done = x, tx_data = xx, tx_done = x,  
10377040000 [MON_RX] DATA = 14, DONE = 1, PUSH = 1, EMPTY = 1  
10377040000 [MON_TX] DATA = e0, DONE = 1, PUSH = 0, EMPTY = 1, FULL = 0  
10377040000 : [tx_data_compare] rx_data = 14, rx_done = x, tx_data = e0, tx_done = 1,  
PASS!!! (Exp: e0, Act: e0)
```

약 4.16ms 뒤에 Pass
약 1.04ms 동안 1개 내보냄
FIFO에서 앞에 3개 전송되고 전송됨

```
-----|  
****  UART + FIFO VERIFICATION  ****  
-----|  
**  Total test cnt      =      256  **  
**  Current Time       =      267658850000  
**  COMPARED cnt       =      253   **  
**  INTERNAL pass cnt  =      256   **  
**  INTERNAL fail cnt  =      0     **  
**  FINAL pass cnt     =      253   **  
**  FINAL fail cnt     =      0     **  
-----|
```

BAUD RATE 가변 검증 목적

- UART는 비동기 시스템이므로, Baud Rate 기반으로 통신
- 같은 9600bps 장비여도, 실제로는 장비마다 약간의 Baud Rate의 차이가 있을 것이라고 생각
- Baud Rate 가 다르더라도 제대로 값을 읽을 수 있는 오차 범위 확인 목적

BAUD RATE 검증 전략

- 이론상 오차율 -3% ~ +3% 까지 허용 가능
- 범위를 3가지 경우로 나누어서 검증
 - -6.25% ~ +6.25%
 - -5.21% ~ +5.21%
 - -4.17% ~ +4.17%
- pass / fail 을 count 해서 모두 pass 되는 범위 확인

-4.17% ~ +4.17%

```
constraint baud_scale_percent {  
    baud_scale inside {[9200:10000]};  
}
```

-5.21% ~ +5.21%

```
constraint baud_scale_percent {  
    baud_scale inside {[9100:11000]};  
}
```

-6.25% ~ +6.25%

```
constraint baud_scale_percent {  
    baud_scale inside {[9000:12000]};  
}
```

BAUD RATE 10,037 (4.37%) : PASS

```
1 190000 : [gen] rx_data = b2, rx_done = x, tx_data = xx, tx_done = x,  
2 190000 : [scb_rx_data] rx_data = b2  
3 196000 : [drv] rx_data = b2, rx_done = x, tx_data = xx, tx_done = x,  
sim: Time (s): cpu = 00:00:02 ; elapsed = 00:00:14 . Memory (MB): peak = 1010.480 ; gain = 0.000
```

```
4 996496000 [DRV] baud_scale = 10037 (= 4.37%), BAUD_sc_per = 99630 ns
```

```
5 1002640000 [MON_RX] DATA = b2, DONE = 1, PUSH = 1, EMPTY = 1
```

```
1002640000 : [gen] rx_data = d4, rx_done = x, tx_data = xx, tx_done = x,
```

```
8269776000 [DRV] baud_scale = 9159 (= -4.41%), BAUD_sc_per = 109180 ns
```

```
9218260000 [MON_TX] DATA = b2, DONE = 1, PUSH = 0, EMPTY = 1, FULL = 0
```

```
9218260000 : [tx_data_compare] rx_data = 56, rx_done = x, tx_data = b2, tx_done = 1,
```

```
PASS!!! (Exp: b2, Act: b2)
```

```
9218260000 : [gen] rx_data = cd, rx_done = x, tx_data = xx, tx_done = x,
```

b2,XX,XX,XX

rx_data = b2 & 오차 4.37% = PASS

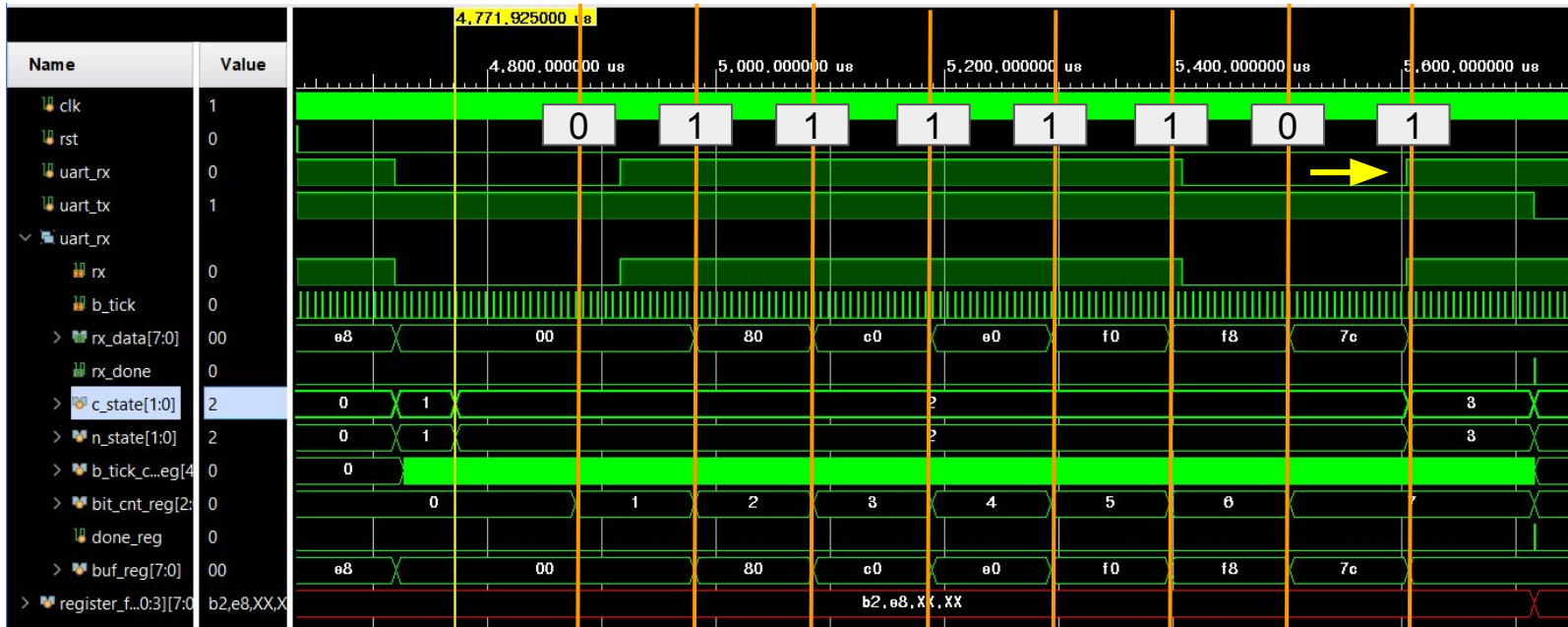
BAUD RATE 10,164 (5.64%) : FAIL

```
1002660000 : [FIFO_DATA_MATCH] fifo_rx_empty = 1, fifo_tx_empty = 0
1 1002660000 : [gen] rx_data = 3e, rx_done = x, tx_data = xx, tx_done = x,
2 1002660000 : [scb_rx_data] rx_data = 3e
4718466000 : [drv] rx_data = 3e, rx_done = x, tx_data = xx, tx_done = x,
5702266000 : [DRV] baud_scale = 10164 (= 5.64%), BAUD_sc_per = 98380 ns
5715880000 : [MON_RX] DATA = be, DONE = 1, PUSH = 1, EMPTY = 1
5715880000 : [gen] rx_data = 1f, rx_done = x, tx_data = xx, tx_done = x,
5715880000 : [scb_rx_data] rx_data = 1f
14530420000 [MON_TX] DATA = be, DONE = 1, PUSH = 0, EMPTY = 1, FULL = 0
14530420000 : [tx_data_compare] rx_data = ed, rx_done = x, tx_data = be, tx_done = 1,
FAIL!!! (Exp:3e, Act:be)
```

b2, a8, be, XX

rx_data = 3e & 오차 5.64% = FAIL

RX Data : 3e , 오차율 5.64%



3e = 0011_1110 / be = 1011_1110 : PC가 보내는 신호가 빨라짐

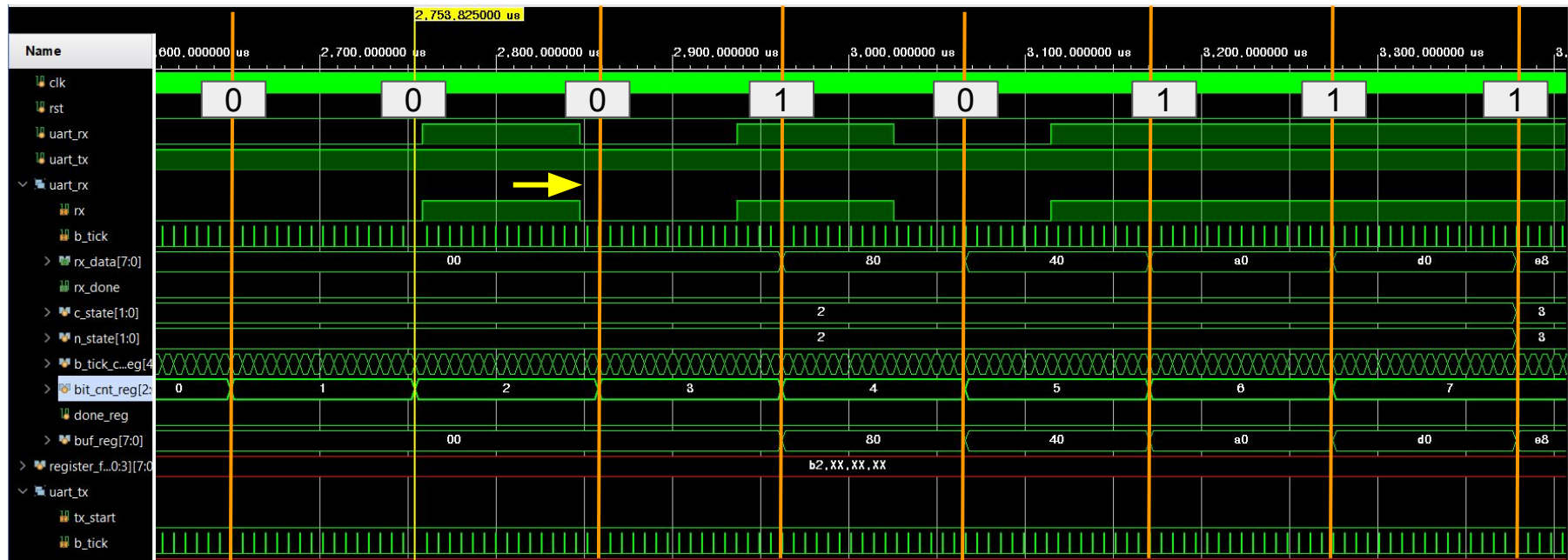
BAUD RATE 11,223 (16.23%) : FAIL

```
1 1002640000 : [gen] rx_data = d4, rx_done = x, tx_data = xx, tx_done = x,
2 1002640000 : [scb_rx_data] rx_data = d4
1002660000 : [scb_pop_data] rx_data = b2, rx_done = 0, tx_data = b2, tx_done = 0,
2044240000 : [scb_rx_data] rx_data = 56
3 2490956000 : [drv] rx_data = d4, rx_done = x, tx_data = xx, tx_done = x,
4 3381956000 [DRV] baud_scale = 11223 (= 16.23%), BAUD_sc_per = 89100 ns
5 3489460000 [MON_RX] DATA = e8, DONE = 1, PUSH = 1, EMPTY = 1
3489460000 : [gen] rx_data = 14, rx_done = x, tx_data = xx, tx_done = x,
11945950000 [MON_TX] DATA = e8, DONE = 1, PUSH = 0, EMPTY = 1, FULL = 0
11945950000 : [tx_data_compare] rx_data = 14, rx_done = x, tx_data = e8, tx_done = 1,
FAIL!!! (Exp:d4, Act:e8)
11945950000 : [gen] rx_data = 4b, rx_done = x, tx_data = xx, tx_done = x,
```

b2, e8, xx, xx

rx_data = d4 & 오차 16.23% = FAIL

RX Data : d4 , 오차율 16.23%



d4 = 1101_0100 / e8 = 1110_1000 : PC가 보내는 신호가 빨라짐

BAUD RATE 한계 분석

통신 속도 증가 or 감소

타이밍 오차가 누적

데이터 오류가 발생

-6.25% ~ +6.25%

```
constraint baud_scale_percent {  
    baud_scale inside {[9000:12000]};  
}
```

```
-----  
****  UART + FIFO VERIFICATION  ****  
-----  
**  Total test cnt      =      256  **  
**  Current Time       =      613190120000  
**  COMPARED cnt       =      253  **  
**  INTERNAL pass cnt   =      256  **  
**  INTERNAL fail cnt   =          0  **  
**  FINAL pass cnt      =      105  **  
**  FINAL fail cnt      =      148  **  
-----
```

-5.21% ~ +5.21%

```
constraint baud_scale_percent {  
    baud_scale inside {[9100:11000]};  
}
```

```
-----  
****  UART + FIFO VERIFICATION  ****  
-----  
**  Total test cnt      =      256  **  
**  Current Time       =      636743300000  
**  COMPARED cnt       =      253  **  
**  INTERNAL pass cnt   =      256  **  
**  INTERNAL fail cnt   =          0  **  
**  FINAL pass cnt      =      148  **  
**  FINAL fail cnt      =      105  **  
-----
```

-4.17% ~ +4.17%

```
constraint baud_scale_percent {  
    baud_scale inside {[9200:10000]};  
}
```

```
-----  
****  UART + FIFO VERIFICATION  ****  
-----  
**  Total test cnt      =      256  **  
**  Current Time       =      667079900000  
**  COMPARED cnt       =      253  **  
**  INTERNAL pass cnt   =      256  **  
**  INTERNAL fail cnt   =          0  **  
**  FINAL pass cnt      =      253  **  
**  FINAL fail cnt      =          0  **  
-----
```

Trouble Shooting : RX Data와 TX Data 비교 FAIL

```
PASS!!!
4127470000 : [scb] rx_data = 95, rx_done = x, tx_data = 61, tx_done = 1
4127470000 : [gen] rx_data = ac, rx_done = x, tx_data = xx, tx_done = x
4166796000 : [drv] rx_data = ad, rx_done = x, tx_data = xx, tx_done = x
5169040000 [MON_RX] DATA : DONE = 1, PUSH = 1, EMPTY = 1
5169040000 : [mon_rx] rx_data = ad, rx_done = 1, tx_data = 61, tx_done = 1
5169040000 : [scb] rx_data = ad, rx_done = 1, tx_data = 61, tx_done = 1
5169040000 : [SCB_PUSH] Data ad | Size: 4
FAIL!!! (Exp:f8, Act:61)
5169040000 : [gen] rx_data = 2b, rx_done = x, tx_data = xx, tx_done = x
5169070000 [MON_TX] DATA : DONE = 0, PUSH = 0, EMPTY = 1, FULL = 0
5169070000 : [mon_tx] rx_data = ad, rx_done = x, tx_data = f8, tx_done = 1
5169070000 : [scb] rx_data = ad, rx_done = x, tx_data = f8, tx_done = 1
FAIL!!! (Exp:04, Act:f8)
```

```
// [RX 모니터]
forever begin
    transaction tr_rx;
    @(posedge uf_if.rx_done);
    #1;
    tr_rx = new;
    @(negedge uf_if.clk); // 샘플링 안정화
    tr_rx.rx_data = uf_if.rx_data;
    tr_rx.tx_data = uf_if.tx_data;
```

```
tr_rx.tx_done = uf_if.tx_done;
```



RX monitor에서 **tr_rx.tx_done**이 계속
1로 나오게 되어, 비교되는 순간이 어긋남

```
// TX data가 왔을 때 꺼내서 비교
if (tr.tx_done) begin
    if (uf_queue.size() > 0) begin
        // Act 값이 xx가 아닐 때만 queue에서 꺼내서 비교
        if (tr.tx_data !== 8'hxx) begin
            compare_data = uf_queue.pop_front();
            if (compare_data === tr.tx_data) $display("PASS!!!");
```

Trouble Shooting : BAUD RATE 오차 범위 초과 PASS 발생

```
//RX data queue 에 넣기
if (tr.rx_done) begin
    uf_queue.push_back(tr.rx_data);
    $display("%t : [SCB_PUSH] Data %h , Size: %d",
        $time, tr.rx_data, uf_queue.size());
end
```

```
fork
    forever begin
        gen2scb_mbox.get(expected_tr);
        uf_queue.push_back(expected_tr.rx_data); // 원본 데이터 저장
        $display("%t : [scb_rx_data] rx_data = %h", $time, expected_tr.rx_data);
    end
join_none
```

tr.rx_data는 PC가 보낸 데이터가 아니라, DUT의 RX에서 온 DATA

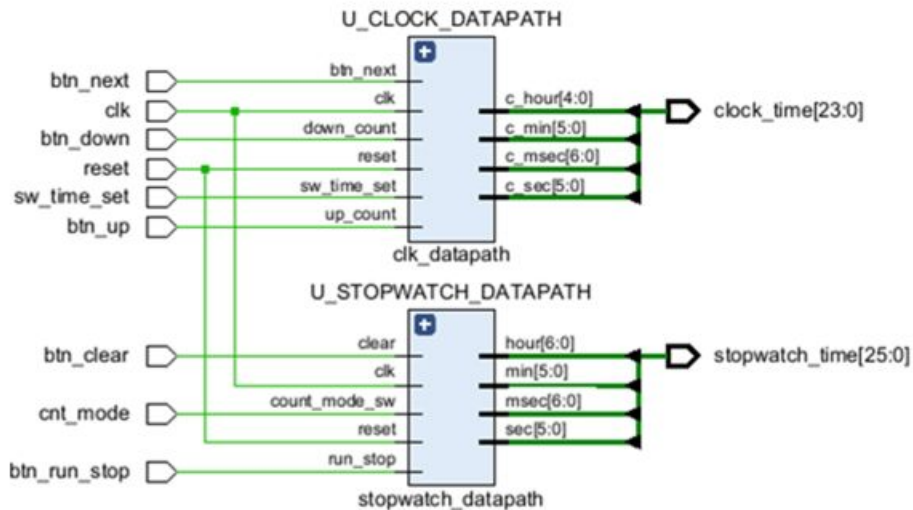
BAUD RATE의 오차가 20%가 되어도 PASS 되는 문제 발생

Stopwatch/Clock

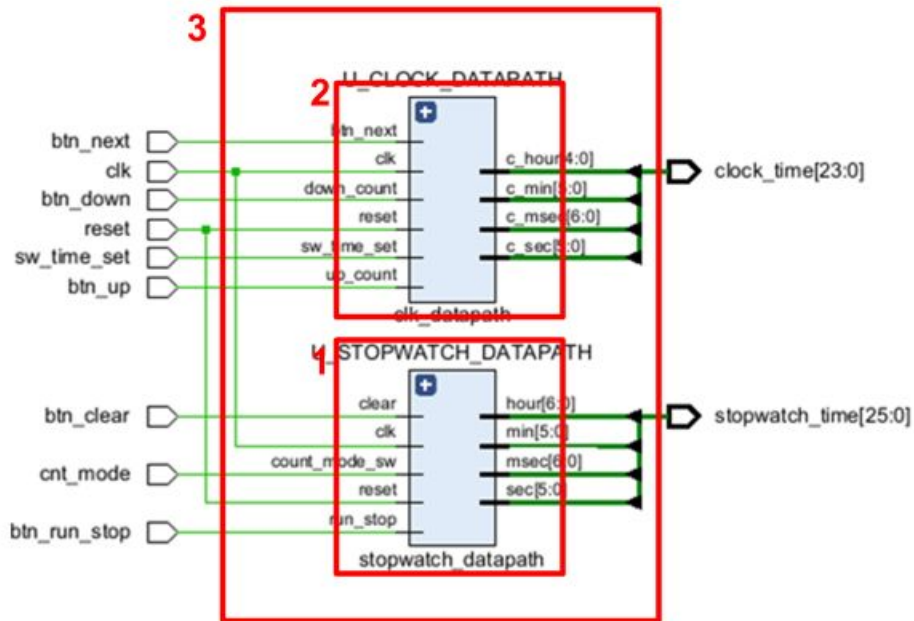
발표자 : 하지훈

Stopwatch/Clock Overview

- 100Mhz 클럭 환경에서 동작하는 시스템
- 스톱워치:
 - 10ms(100Hz) 단위 카운트
 - Up/Down 카운트 모드
 - Run/Stop/Clear 기능
 - 00:00:00.00 ~ 99:59:59.99
- 디지털 시계:
 - 10ms(100Hz) 단위 카운트
 - 시간 설정 모드(Time set) - Up/Down/Next sel
 - 00:00:00.00 ~ 23:59:59.99

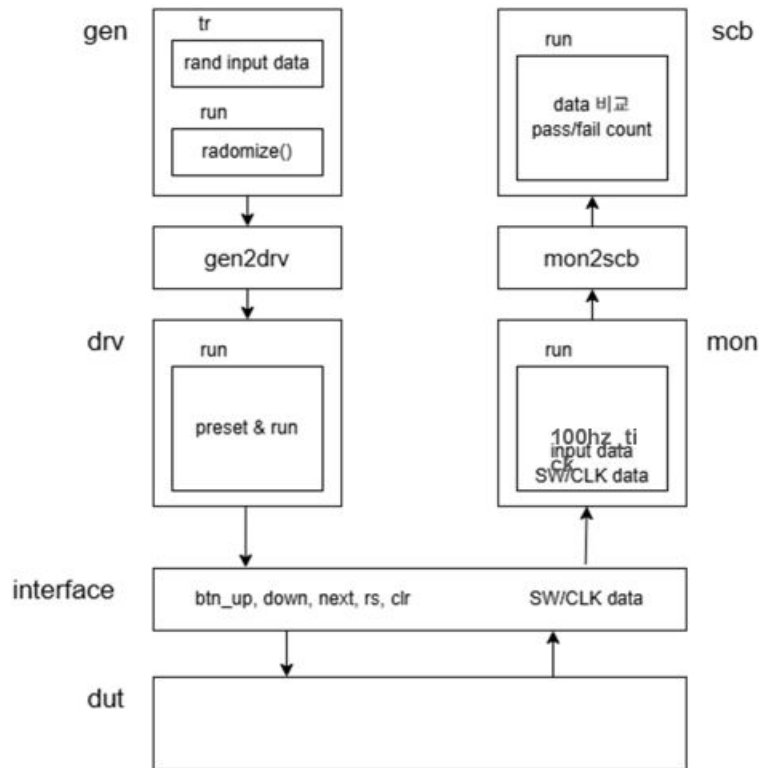


Verification Flow



Testbench Block

블록	동작
gen	랜덤 입력 생성
drv	신호, 리셋 제어
Mon	Posedge+1ns Data Sampling
Scb	- DUT의 100Hz Tick 기반 레퍼런스 모델 - 100hz tick 및 이벤트 발생 시 DUT 출력 값을 캡처 - Pass/Fail 판정
interface	신호 연결



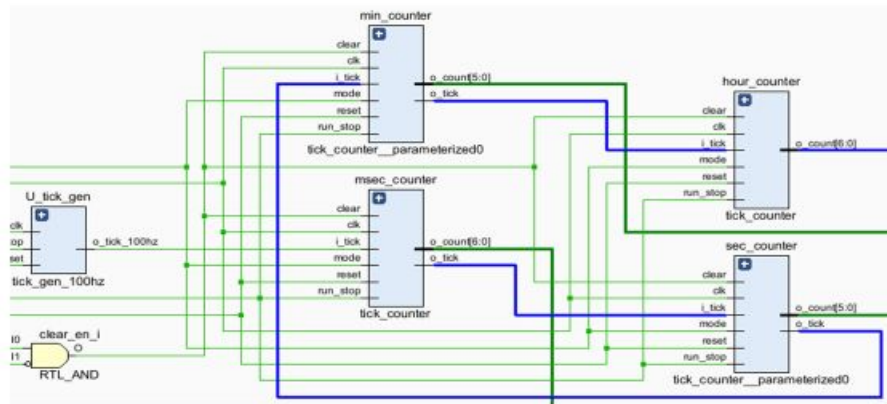
- Dut의 출력 값 / 레퍼런스 모델이 산출한 exp data를 대조하여 Pass/Fail 판정

** **Pass** 의 경우 이벤트 발생 시에만 카운트

** **Fail** 의 경우 이벤트 유무와 상관없이 모든 불일치(Mismatch) 발생시 카운트(100Hz 샘플링)

Design Details - Stopwatch

- Tick Gen: 100MHz 클럭을 분주하여 100Hz 펄스 생성
- 하위 카운터 캐리를 상위 카운터로 전파하는 구조
- Run/Stop: 단일 버튼 토글 방식 사용
- Clear: 동시 입력 충돌 방지 설계



Verification Test Scenario

- Test 1

- 1) Run/Stop, Clear 버튼 동작 확인
- 2) Clear 버튼 동작 조건 확인

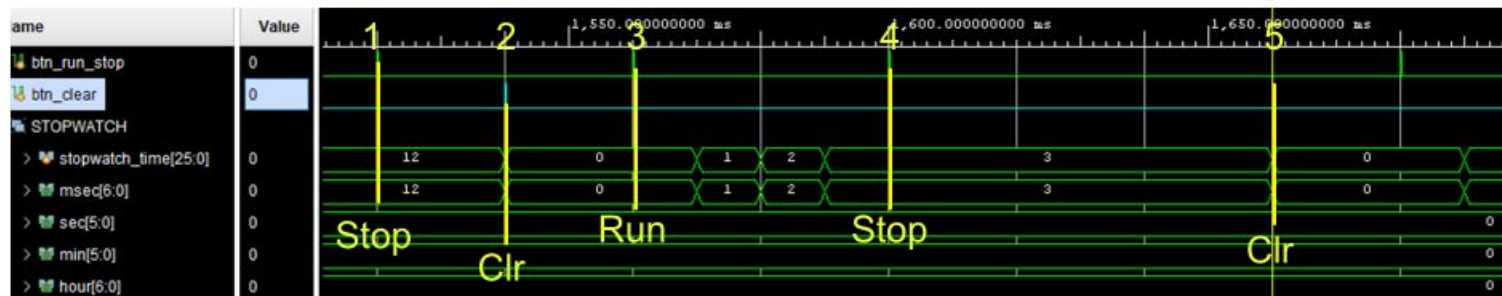
- Test 2

- 1) Count Mode 변경 테스트
- 2) 롤오버 테스트 (00:00:00.00 -> 99:59:59.99 -> 00:00:00.00)

- Test 3

- Full case 테스트

Verification Test 1.1



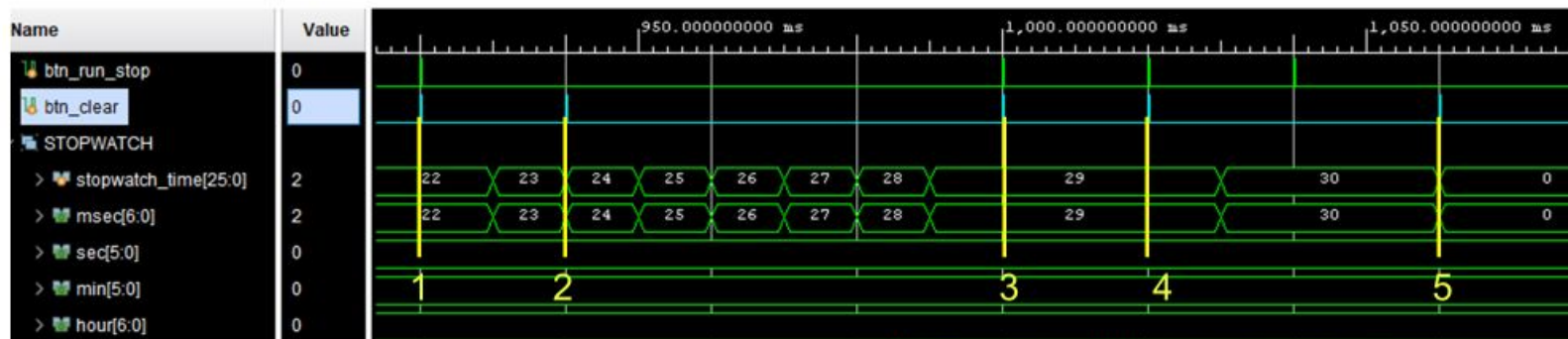
● Run/Stop, Clear 버튼 입력 테스트

- 입력 확률: run/stop = 50%, clear = 20%
- 입력 간격: 20ms
- 총 테스트 타임 2s

```
1 15200000026000 : [SW][PASS] ref=000000c dut=000000c | rs=1 clr=0
2 15400000026000 : [SW][PASS] ref=0000000 dut=0000000 | rs=0 clr=1
3 15600000026000 : [SW][PASS] ref=0000000 dut=0000000 | rs=1 clr=0
4 16000000026000 : [SW][PASS] ref=0000003 dut=0000003 | rs=1 clr=0
5 16600000026000 : [SW][PASS] ref=0000000 dut=0000000 | rs=0 clr=1
  16800000026000 : [SW][PASS] ref=0000000 dut=0000000 | rs=1 clr=0
  17000000026000 : [SW][PASS] ref=0000001 dut=0000001 | rs=1 clr=0
  ===== SUMMARY =====
  STOPWATCH: PASS=73 FAIL=0
  BTN COUNT: run_stop=54 clear=19
```

-> Run/Stop 동작 및 Clear 동작 확인

Verification Test 1.2



● Clear 버튼 동작 조건 확인

- 입력 확률: run/stop = 40%, clear = 15%
- 입력 간격: 20ms
- 총 테스트 타임 2s

```

1 920000026000 : [SW][PASS] ref=0000016 dut=0000016 | rs=1 clr=1
2 940000026000 : [SW][PASS] ref=0000017 dut=0000017 | rs=0 clr=1
3 1000000026000 : [SW][PASS] ref=000001d dut=000001d | rs=1 clr=1
4 1020000026000 : [SW][PASS] ref=000001d dut=000001d | rs=1 clr=1
  1040000026000 : [SW][PASS] ref=000001e dut=000001e | rs=1 clr=0
5 1060000026000 : [SW][PASS] ref=0000000 dut=0000000 | rs=0 clr=1
  
```

Case 1,3)Run 상태에서 동시 입력 발생 -> clr 무시

Case 2)Run 상태에서(rs=0) clr 입력 무시

Case 4)Stop 상태에서 동시 입력 발생 -> Run 상태 진입

Case 5)Stop 상태에서(rs=0) clr 입력 -> clr 동작

===== SUMMARY =====

STOPWATCH: PASS=58 FAIL=0

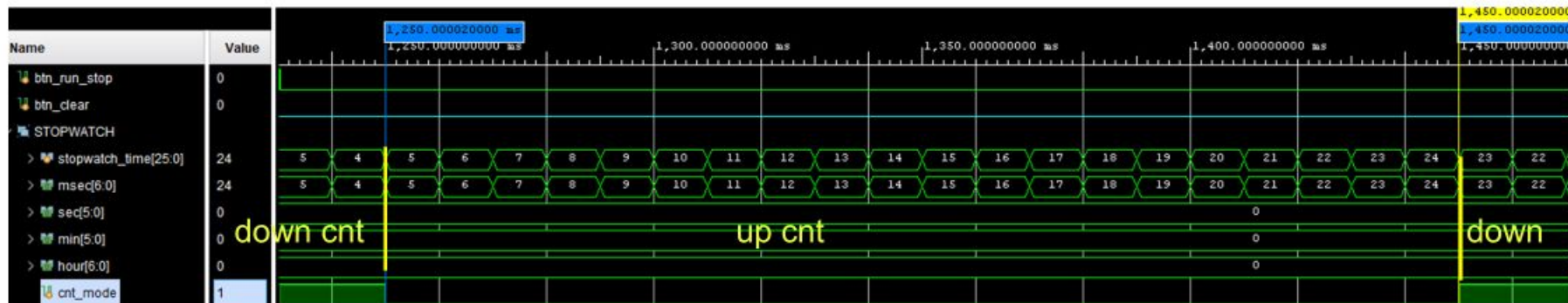
BTN COUNT: run_stop=46 clear=22

** rs, clr 신호가 동시에 들어온 경우 Pass/fail은 1회로 카운트 됨

-> 입력 우선 순위 Run/Stop > Clear인 것을 확인
(동시 입력 시 Clear 무시)

```
wire clear_en = clear & ~run_en & ~run_stop;
```

Verification Test 2.1



● Count Mode 변경 테스트

- 토글 간격: 200ms
- 총 테스트 타임 2s

```
450000026000 : [SW][PASS] ref=0000004 dut=0000004 | mode=0
650000026000 : [SW][PASS] ref=0000018 dut=0000018 | mode=1
850000026000 : [SW][PASS] ref=0000004 dut=0000004 | mode=0
1050000026000 : [SW][PASS] ref=0000018 dut=0000018 | mode=1
1250000026000 : [SW][PASS] ref=0000004 dut=0000004 | mode=0
1450000026000 : [SW][PASS] ref=0000018 dut=0000018 | mode=1
===== SUMMARY =====
STOPWATCH: PASS=11 FAIL=0
SW TOGGLE: cnt_mode=10
** Run동작 Pass 신호 +1 발생
```

-> Mode = 1 down count/ Mode = 0 up count 확인

Verification Test 2.2



● 롤오버 테스트

- 토글 간격 : 200ms
- 총 테스트 타임 2s

● 초기 모드 설정

- 99:59:59:10 시작(down 카운트를 이용하여 설정)
- 00:00:00.00 -> 99:59:59.99 -> 00:00:00.00 경계 구간 반복

```
100000026000 : [SW] [PASS] ref=0000000 dut=0000000 | mode=1
200000026000 : [SW] [PASS] ref=31f7ddb dut=31f7ddb | mode=0
400000026000 : [SW] [PASS] ref=000000b dut=000000b | mode=1
600000026000 : [SW] [PASS] ref=31f7ddb dut=31f7ddb | mode=0
800000026000 : [SW] [PASS] ref=000000b dut=000000b | mode=1
100000026000 : [SW] [PASS] ref=31f7ddb dut=31f7ddb | mode=0
120000026000 : [SW] [PASS] ref=000000b dut=000000b | mode=1
```

===== SUMMARY =====

STOPWATCH: PASS=12 FAIL=0

SW TOGGLE: cnt_mode=11

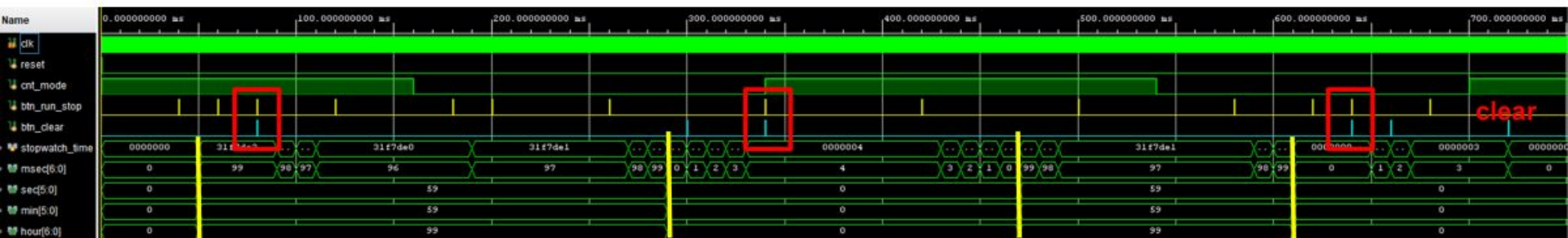
** Run동작 Pass 신호 +1 발생

-> 롤오버 정상 동작 확인

Verification Strategy - Test 3 Full case

- 이벤트 발생 확률 랜덤화
 - 입력 확률: run/stop = 53%, clear = 48%
 - 입력 간격: 20ms
 - 카운트 방향: up/down 랜덤 토글(100 ~ 300ms)
 - Coverage를 통한 버튼 개별 입력 및 스위치 조합 케이스 확인
 - (run/stop), (clear), (run/stop | clear), (cnt_mode), (cnt_mode) | btn
- 초기 모드 설정
 - Down-count 모드로 시작 - 경계 구간에서 롤오버를 유도하기 위함
(00:00:00.00 -> 99:59:59.99 -> 00:00:00.00)

Verification Checked - Test 3 Full case

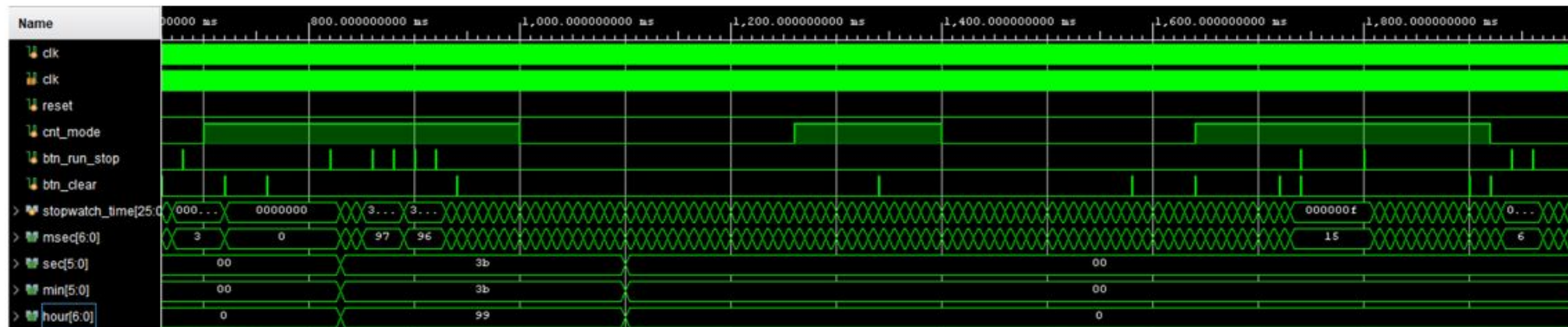


```

40000026000 : [SW][PASS] ref=0000000 dut=0000000 | mode=1 rs=1 clr=0
60000026000 : [SW][PASS] ref=31f7de3 dut=31f7de3 | mode=1 rs=1 clr=0
80000026000 : [SW][PASS] ref=31f7de3 dut=31f7de3 | mode=1 rs=1 clr=1
120000026000 : [SW][PASS] ref=31f7de0 dut=31f7de0 | mode=1 rs=1 clr=0
340000026000 : [SW][PASS] ref=0000004 dut=0000004 | mode=1 rs=1 clr=1
620000026000 : [SW][PASS] ref=0000000 dut=0000000 | mode=0 rs=1 clr=0
640000026000 : [SW][PASS] ref=0000000 dut=0000000 | mode=0 rs=1 clr=1
660000026000 : [SW][PASS] ref=0000001 dut=0000001 | mode=0 rs=0 clr=1
680000026000 : [SW][PASS] ref=0000003 dut=0000003 | mode=0 rs=1 clr=0
700000026000 : [SW][PASS] ref=0000003 dut=0000003 | mode=1 rs=0 clr=0
720000026000 : [SW][PASS] ref=0000000 dut=0000000 | mode=1 rs=0 clr=1 clear
    
```

- R/S, Clr 동시 입력 발생
-> Clr 비활성 상태 유지
- 경계 구간
-> 정상적 롤오버 확인

Verification Checked - Test 3 Full case



===== SUMMARY =====

STOPWATCH: PASS=51 FAIL=0

BTN COUNT: run_stop=23 clear=15

SW TOGGLE: cnt_mode=9

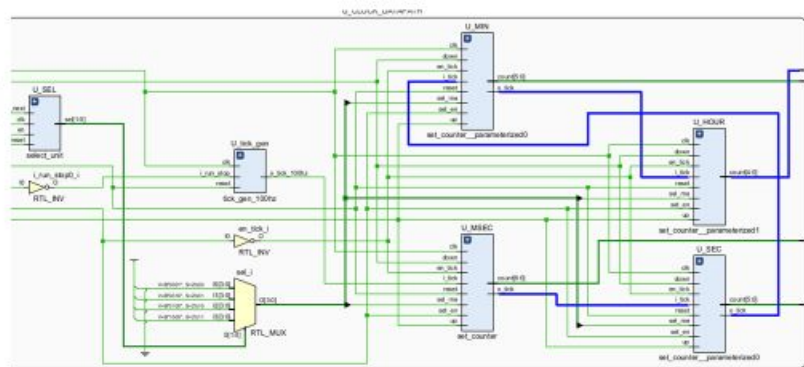
COVERAGE : 100.00%

time : 2000000215 ns

- Run/Stop 토글 및 상태 유지 정상 동작 확인
- Clear 정상 동작 확인
- 버튼 동시 입력 정상 동작 확인
- 경계 구간 롤오버 (99->00, 00 ->99) 정상 동작 확인
- 사용자 조작 시나리오 전수 검증 완료 (coverage = 100%)

Design Details - Clock

- Tick Gen: 100MHz 클럭을 분주하여 100Hz 펄스 생성
- 하위 카운터 캐리를 상위 카운터로 전파하는 구조
- Time-set 모드 시 시간 정지
- Up/Down 버튼: 동시 입력 충돌 방지 설계
- Next 버튼: 시간 변경 자리 선택 순환



Verification Test Scenario

- Test 1

- 1) Time-set 모드 진입 시 시간 정지 확인
- 2) UP/DOWN/Next 버튼 시간 변경 동작 확인
- 3) UP/DOWN/Next 동시 입력 동작 확인

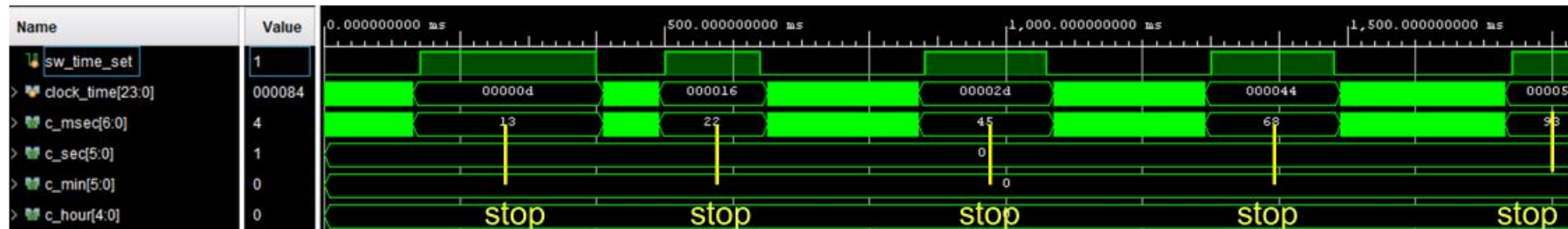
- Test 2

- 롤오버 테스트 00:00:00.00 -> 23:59:59.99 -> 00:00:00.00

- Test 3

- Full case

Verification Test 1.1



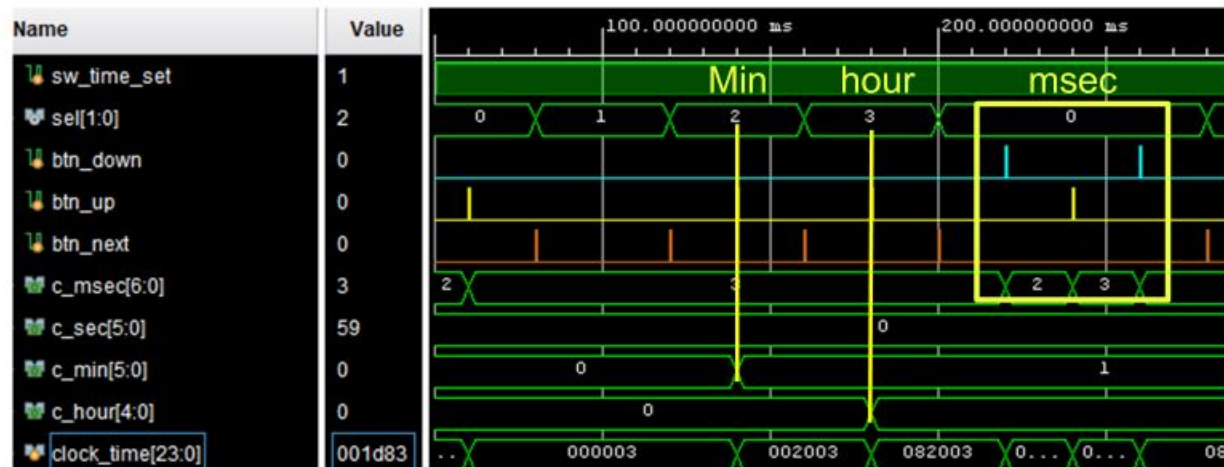
● Time-set mode 테스트

- Mode 토글 주기: (100 ~ 300ms)
- 총 테스트 타임 2s

```
1400000026000 : [CLK] [PASS] ref=00000d dut=00000d | tset=1
4000000026000 : [CLK] [PASS] ref=00000d dut=00000d | tset=0
5000000026000 : [CLK] [PASS] ref=000016 dut=000016 | tset=1
6400000026000 : [CLK] [PASS] ref=000016 dut=000016 | tset=0
8800000026000 : [CLK] [PASS] ref=00002d dut=00002d | tset=1
===== SUMMARY =====
CLOCK      : PASS=11 FAIL=0
SW TOGGLE: sw_time_set=11
```

-> Time-set mode 진입 시 시간 정지 확인

Verification Test 1.2



● Up/Down/Next 버튼 시간 변경 확인

- Up, Down, Next = 25%
- 입력 간격 20ms
- 총 테스트 타임 2s

```
60000026000 : [CLK] [PASS] ref=000003 dut=000003 | tset=1 up=1 down=0 next=0
80000026000 : [CLK] [PASS] ref=000003 dut=000003 | tset=1 up=0 down=0 next=1
10000026000 : [CLK] [PASS] ref=000003 dut=000003 | tset=1 up=0 down=0 next=0
12000026000 : [CLK] [PASS] ref=000003 dut=000003 | tset=1 up=0 down=0 next=1
14000026000 : [CLK] [PASS] ref=002003 dut=002003 | tset=1 up=1 down=0 next=0
16000026000 : [CLK] [PASS] ref=002003 dut=002003 | tset=1 up=0 down=0 next=1
```

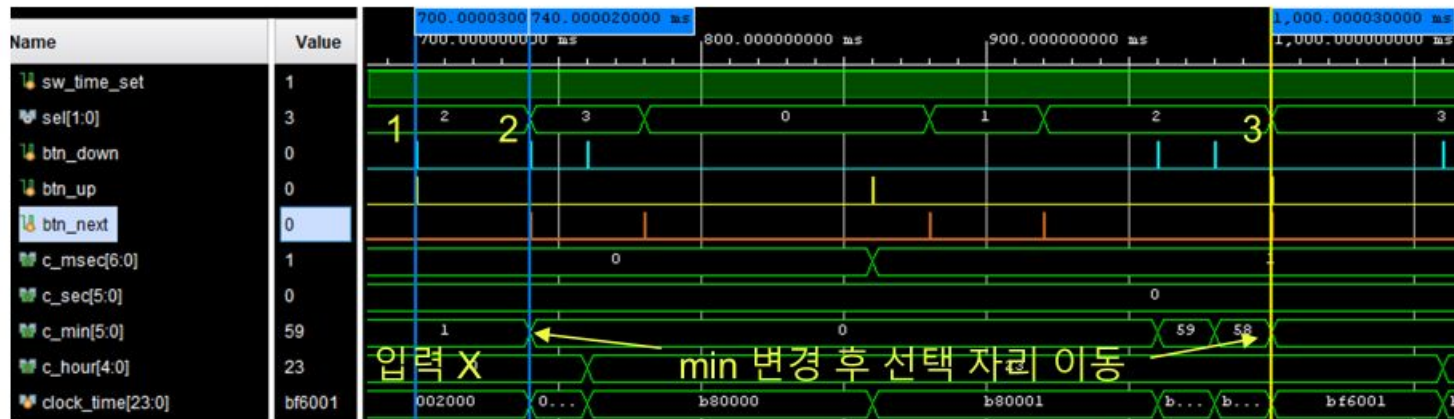
===== SUMMARY =====

CLOCK : PASS=100 FAIL=0

BTN COUNT: up=35 down=17 next=35

-> Time-set mode 시간 변경 기능 확인

Verification Test 1.3



● 동시입력 동작 확인

- Up, Down, Next = 25%
- 총 테스트 타임 2s

===== SUMMARY =====

CLOCK : PASS=63 FAIL=0

BTN COUNT: up=27 down=30 next=22

```
SW TOGGLE: sw time set=1
```

```
1 7000000026000 : [CLK][PASS] ref=002000 dut=002000 | tset=1 up=1 down=1 next=0
```

```
2 7400000026000 : [CLK][PASS] ref=000000 dut=000000 | tset=1 up=0 down=1 next=1
```

```
760000026000 : [CLK][PASS] ref=b80000 dut=b80000 | tset=1 up=0 down=1 next=0
```

```
3 10000000026000 : [CLK][PASS] ref=bf6001 dut=bf6001 | tset=1 up=1 down=0 next=1
```

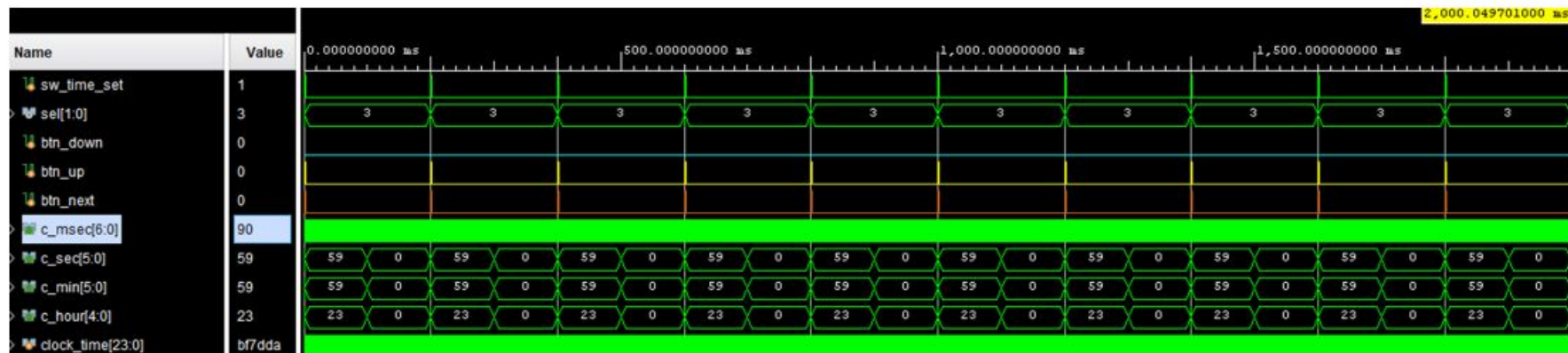
Case 1) down&&up 버튼 입력 무시됨

Case 2) down&&next down 버튼 선 입력

Case 3) up&&next up 버튼 선 입력

-> Up, Down 우선 입력, Up&&Down 동시 입력 무시
의도한대로 동작하는 것을 확인

Verification Test 2



● 룰오버 테스트

- 총 테스트 타임 2s
- 초기 시간 세팅 23:59:59:90 -> 00:00:00:10 -> 23:59:59:90 반복 (Time-set 모드 활용)

```

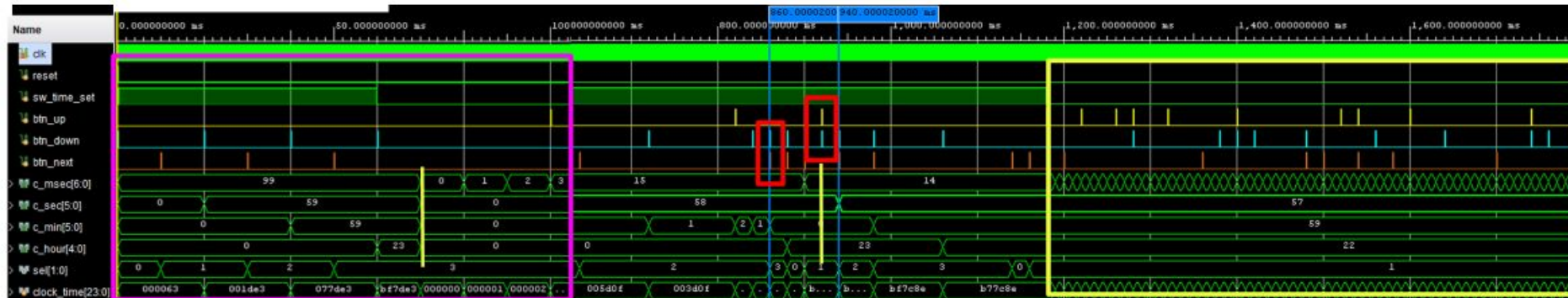
1400031636000 : [Pass 7] clock_time=00000a (0:0:0:10)
1400036131000 : [Time-set] clock_time=bf7dda (23:59:59:90)
1600036126000 : [Pass 8] clock_time=00000a (0:0:0:10)
1600040621000 : [Time-set] clock_time=bf7dda (23:59:59:90)
1800040616000 : [Pass 9] clock_time=00000a (0:0:0:10)
1800045111000 : [Time-set] clock_time=bf7dda (23:59:59:90)
2000045106000 : [Pass 10] clock_time=00000a (0:0:0:10)
2000049601000 : [Time-set] clock_time=bf7dda (23:59:59:90)
2000049601000 : Total_Time-set=10
    
```

-> 경계 구간 정상 동작 확인

Verification Strategy – Test 3 Full case

- 이벤트 발생 확률 랜덤화
 - 입력 확률: up, down, next $\approx$ 35%
 - 입력 간격: 20ms
 - Time-set 모드: 랜덤 토글(100 ~ 200ms)
 - Coverage를 통한 버튼 개별 입력 및 스위치 조합 케이스 확인
- 초기 모드 설정
 - Time-set = 23:59:59.99 - 롤오버 구간을 빠르게 확인하기 위함
(00:00:00.00 -> 23:59:59.99 -> 00:00:00.00)

Verification Checked – Test 3 Full case



```

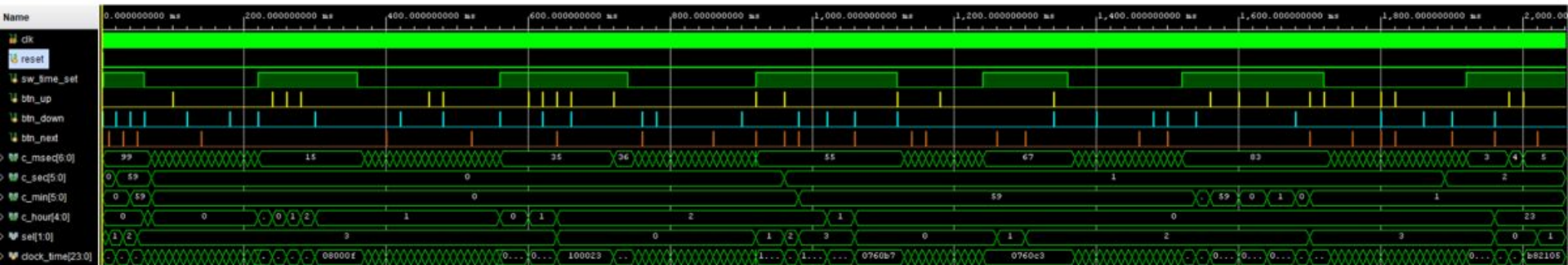
10000036000 : [CLK] [PASS] ref=000063 dut=000063 | tset=1 up=0 down=0 next=1
20000046000 : [CLK] [PASS] ref=001de3 dut=001de3 | tset=1 up=0 down=1 next=0
30000056000 : [CLK] [PASS] ref=001de3 dut=001de3 | tset=1 up=0 down=0 next=1
40000066000 : [CLK] [PASS] ref=077de3 dut=077de3 | tset=1 up=0 down=1 next=0
50000076000 : [CLK] [PASS] ref=077de3 dut=077de3 | tset=1 up=0 down=0 next=1
60000086000 : [CLK] [PASS] ref=bf7de3 dut=bf7de3 | tset=1 up=0 down=1 next=0
60000096000 : [CLK] [PASS] ref=bf7de3 dut=bf7de3 | tset=0 up=0 down=0 next=0
860000026000 : [CLK] [PASS] ref=001d0f dut=001d0f | tset=1 up=0 down=1 next=1
880000026000 : [CLK] [PASS] ref=b81d0f dut=b81d0f | tset=1 up=0 down=1 next=1
900000026000 : [CLK] [PASS] ref=b81d0e dut=b81d0e | tset=1 up=0 down=1 next=1
920000026000 : [CLK] [PASS] ref=b81d0e dut=b81d0e | tset=1 up=1 down=1 next=0
940000026000 : [CLK] [PASS] ref=b81c8e dut=b81c8e | tset=1 up=0 down=1 next=1
980000026000 : [CLK] [PASS] ref=bf7c8e dut=bf7c8e | tset=1 up=0 down=1 next=1
    
```

- sel: 선택 자리(00, 01, 10, 11)
- Up/Down 동시 입력 시 입력 무시 동작 확인
- 경계 구간 롤오버 동작 확인
- Time-set 모드 정지 확인

min 변경

시간 변경 안됨

Verification Checked – Test 3 Full case



===== SUMMARY =====

CLOCK : PASS=49 FAIL=0

BTN COUNT: up=26 down=29 next=26

SW TOGGLE: sw time set=12

COVERAGE : 100.00%

time : 2000000190 ns

- Time-set mode = 0 버튼 입력 무시 확인
- Time-set mode = 1 시간 변경 정상 동작 확인
- Next 버튼 동작 확인
- 경계 구간 롤오버 정상 동작 확인
- 사용자 조작 시나리오 전수 검증 완료 (coverage = 100%)

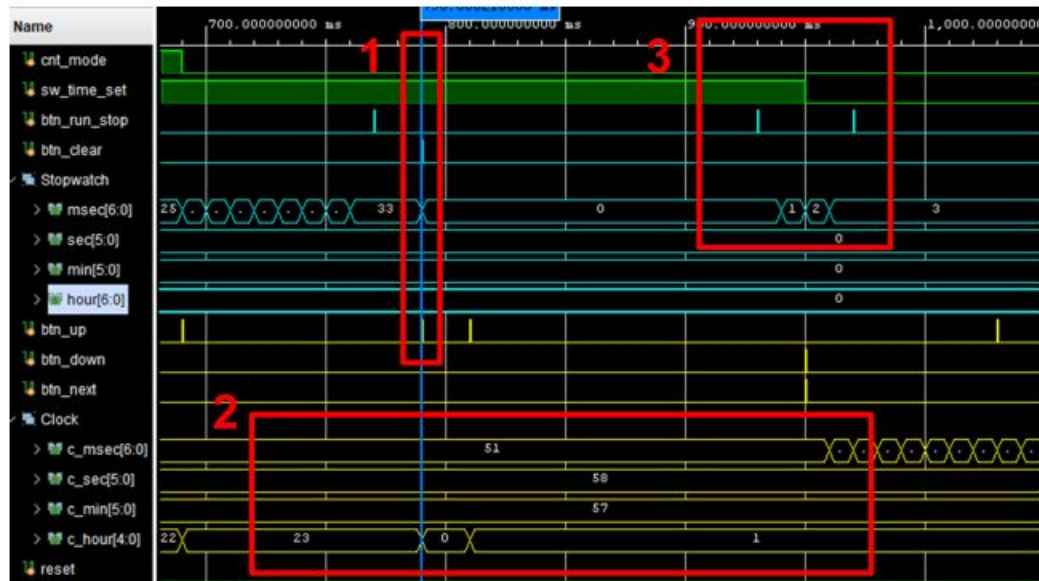
Integration Verification

Integration Verification Strategy

- 각 모듈의 독립성 확인
 - 공유 시스템 Clk으로 stopwatch, clock 동시 동작 시 데이터 및 독립성 확인
- 이벤트 발생 확률 랜덤화
 - 모드 기반 가변 확률 적용
: Time-set 모드에서는 시간 설정 버튼(up/down/next) 입력 확률 증가
 - 입력 간격: 20ms
 - 카운트 방향: up/down 랜덤 토글(100 ~ 500ms)
 - Time-set 모드 랜덤 토글 (200 ~ 1000ms)
 - Coverage를 통한 버튼 개별 입력 및 스위치 조합 케이스 확인

```
constraint c_btn_5pct {  
    if (sw_time_set) {  
        btn_run_stop dist {1 := 11, 0 := 89};  
        btn_clear    dist {1 := 8, 0 := 92};  
        btn_up       dist {1 := 9, 0 := 91};  
        btn_down     dist {1 := 9, 0 := 91};  
        btn_next     dist {1 := 11, 0 := 89};  
    } else {  
        btn_run_stop dist {1 := 10, 0 := 90};  
        btn_clear    dist {1 := 4, 0 := 96};  
        btn_up       dist {1 := 5, 0 := 95};  
        btn_down     dist {1 := 5, 0 := 95};  
        btn_next     dist {1 := 5, 0 := 95};  
    }  
}
```

Verification Checked

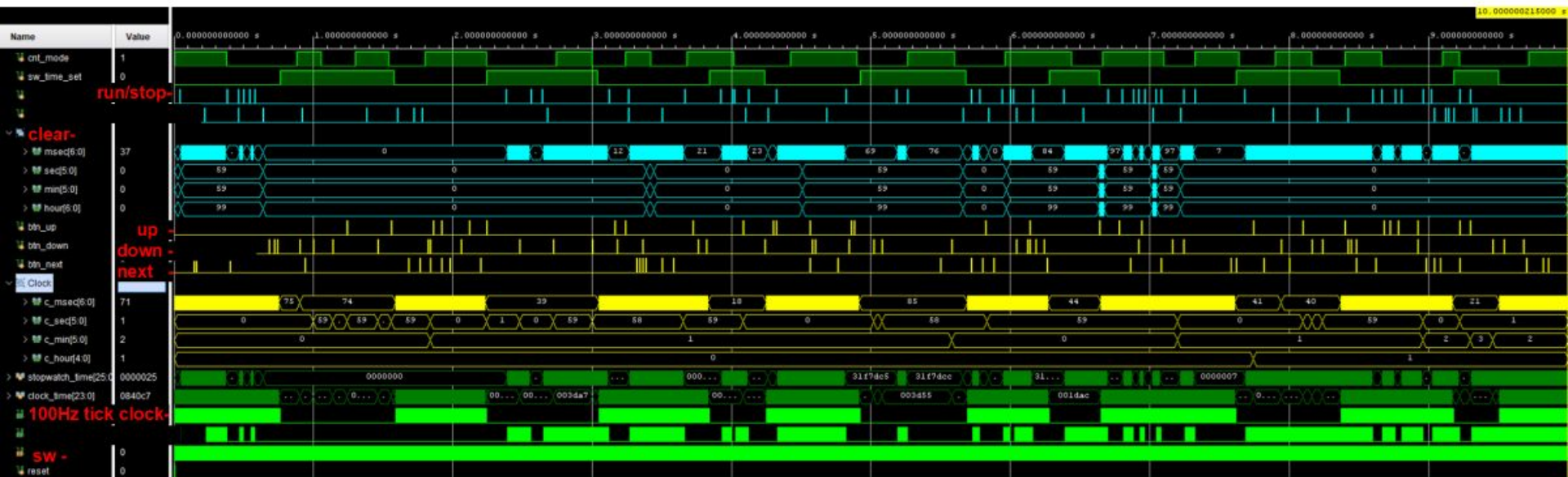


Case1:
모듈 간 동시 입력 시 간섭
없음 확인

Case2:
clock time-set 동작 확인

Case3:
Run/Stop 동작 확인

Verification Checked



===== SUMMARY =====

STOPWATCH: PASS=195 FAIL=0

CLOCK : PASS=60 FAIL=0

BTN COUNT: run_stop=45 clear=35 up=29 down=42 next=3

SW TOGGLE: cnt_mode=30 sw_time_set=14

COVERAGE : 100.00%

```
7820000026000 : [SW][PASS] ref=0000014 dut=0000014 [CLK][PASS] ref=082029 dut=082029
mode=0 tset=1 rs=0 clr=0 up=0 down=0 next=1
7880000026000 : [SW][PASS] ref=000001a dut=000001a [CLK][PASS] ref=082029 dut=082029
mode=0 tset=1 rs=0 clr=1 up=0 down=0 next=0
7900000026000 : [SW][PASS] ref=000001c dut=000001c [CLK][PASS] ref=082029 dut=082029
mode=1 tset=1 rs=0 clr=0 up=0 down=0 next=0
7940000026000 : [SW][PASS] ref=0000018 dut=0000018 [CLK][PASS] ref=082028 dut=082028
mode=1 tset=1 rs=0 clr=0 up=0 down=1 next=0
```

Verification Checked

- 검증 결과
 - Stopwatch/Clock 정상 동작 (경계 구간 롤오버 정상 동작 확인)
 - Mode 전환 정상 동작
 - Time-set 정상 동작
 - 동시 입력 및 버튼 간섭 없음
 - 사용자 조작 시나리오 전수 검증 완료 (coverage = 100%)

```
===== SUMMARY =====  
STOPWATCH: PASS=195  FAIL=0  
CLOCK      : PASS=60  FAIL=0  
BTN COUNT: run_stop=45  clear=35  up=29  down=42  next=38  
SW TOGGLE: cnt_mode=30  sw_time_set=14  
COVERAGE  : 100.00%
```

Fault Injection Test

```
assign stopwatch_if.stopwatch_time = sw_error ^ 26'd1;
assign stopwatch_if.clock_time     = clk_error ^ 24'd1;
```

clk_error[23:0]	6	2	3	4	5	6	7	8	9	10	11
clock_time[23:0]	7	3	2	5	4	7	6	9	8	11	10
sw_error[25:0]	2		0		1	2	3	4	5	6	7
stopwatch_time[25:0]	3		1		0	3	2	5	4	7	6

```
180000026000 : [SW][FAIL] ref=000000d dut=000000c [CLK][FAIL] ref=000011 dut=000013
               mode=0 tset=0 rs=0 clr=0 up=1 down=0 next=0
200000026000 : [SW][FAIL] ref=000000f dut=000000e [CLK][FAIL] ref=000013 dut=000015
               mode=0 tset=0 rs=0 clr=0 up=1 down=0 next=0
220000026000 : [SW][FAIL] ref=0000011 dut=0000010 [CLK][FAIL] ref=000015 dut=000017
               mode=0 tset=0 rs=0 clr=1 up=0 down=0 next=0
880000026000 : [SW][FAIL] ref=31f7ddf dut=31f7dde [CLK][FAIL] ref=00004b dut=00004a
               mode=0 tset=1 rs=0 clr=0 up=0 down=0 next=0
900000026000 : [SW][FAIL] ref=31f7ddf dut=31f7dde [CLK][FAIL] ref=00004b dut=00004a
               mode=0 tset=1 rs=1 clr=0 up=1 down=1 next=0
920000026000 : [SW][FAIL] ref=31f7de0 dut=31f7de1 [CLK][FAIL] ref=00004b dut=00004a
               mode=0 tset=1 rs=1 clr=0 up=0 down=0 next=0
940000026000 : [SW][FAIL] ref=31f7de0 dut=31f7de1 [CLK][FAIL] ref=00004b dut=00004a
               mode=0 tset=1 rs=1 clr=0 up=0 down=0 next=1

===== SUMMARY =====
STOPWATCH: PASS=0  FAIL=37
CLOCK      : PASS=0  FAIL=37

BIN COUNT: run_stop=12 clear=5 up=21 down=5 next=4
SW TOGGLE: cnt_mode=2 sw_time_set=1
```

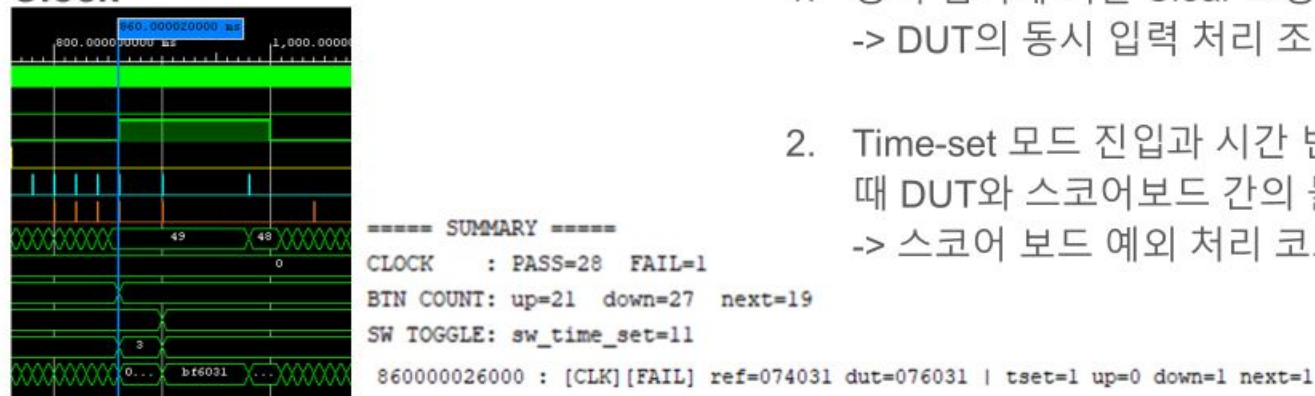
- Scoreboard의 오류 검출 능력 확인
- monitor가 캡처하는 stopwatch_if 출력의 LSB 1bit를 반전
- 로그에서 스코어보드의 데이터 검증 불일치 확인

Troubleshooting

Stopwatch



Clock



1. 동시 입력에 의한 Clear 오동작 확인
-> DUT의 동시 입력 처리 조건 개선
2. Time-set 모드 진입과 시간 변경 펄스 동시 인가되었을 때 DUT와 스코어보드 간의 불일치 발생
-> 스코어 보드 예외 처리 코드 추가

감사합니다.